

Deep Learning

Summary

“Neurons that fire together wire together”

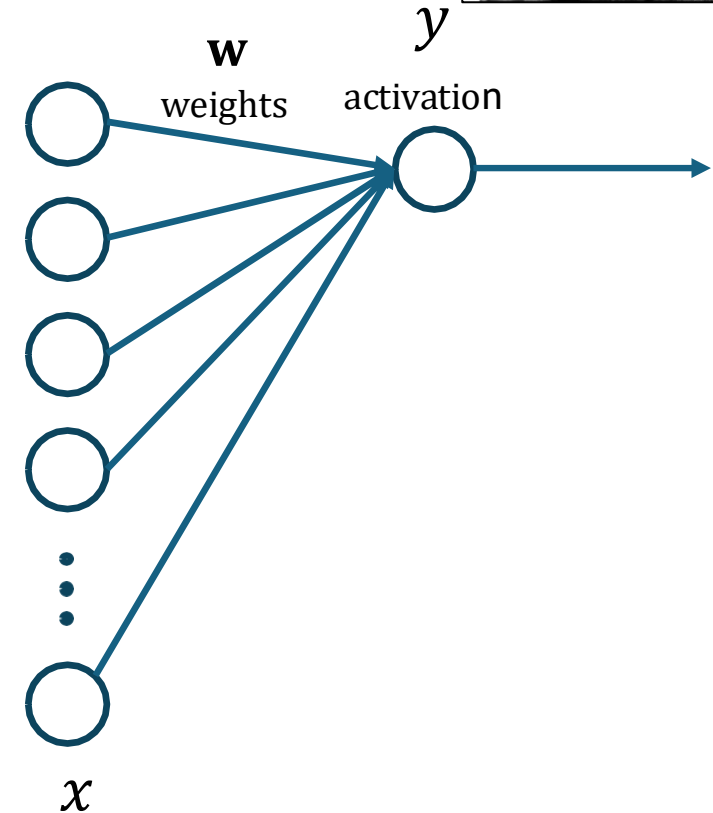


Hebbian Learning

- Let $x \in \mathbb{R}^d$ a feature vector and weights $w \in \mathbb{R}^d$
- Output is linear: $y = w^\top x$, $y \in \mathbb{R}$
- We change the weights according to the following rule:

$$w_{t+1} = w_t + xy$$

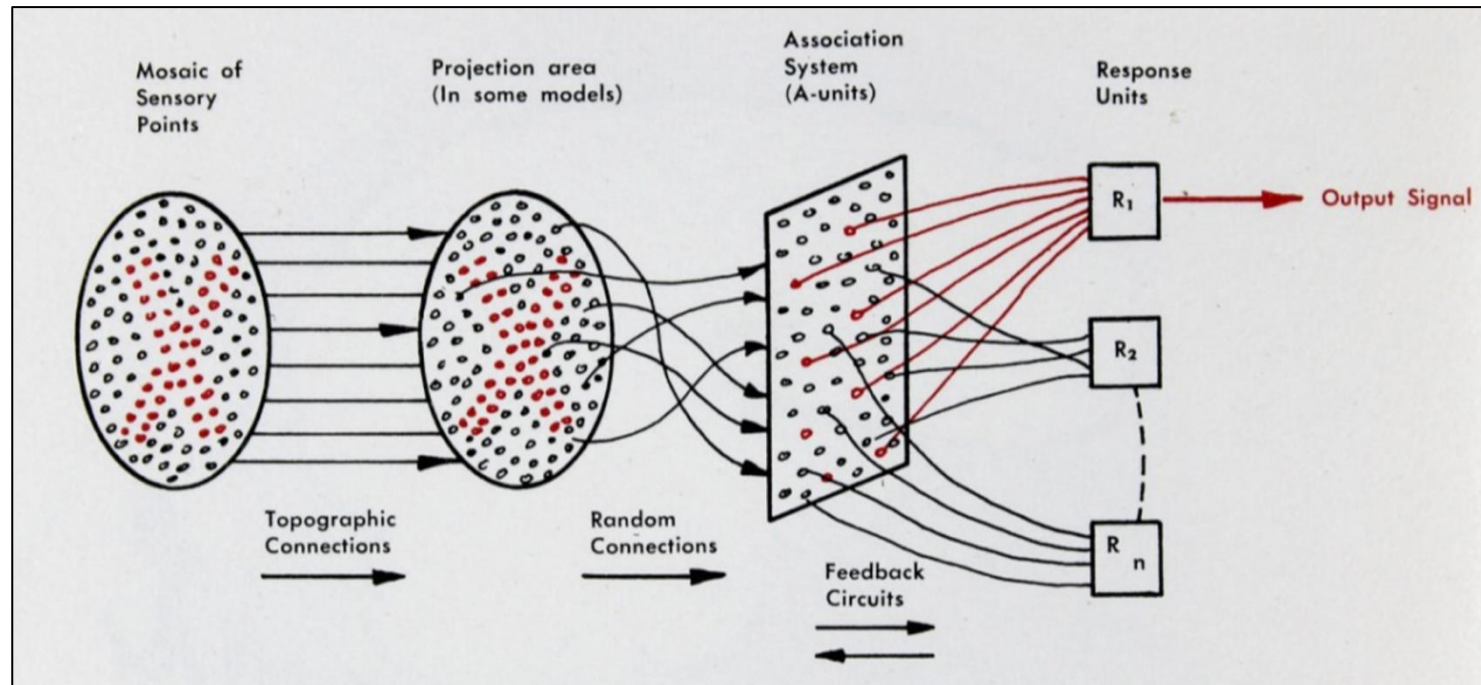
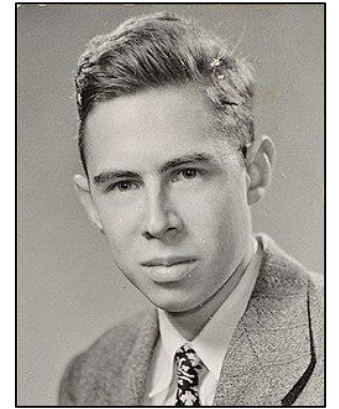
- Could learn online PCA (w/ Oja's Rule*)



* https://en.wikipedia.org/wiki/Oja%27s_rule

Rosenblatt's Perceptron (1958)

- **Learns from labels!**



The Perceptron

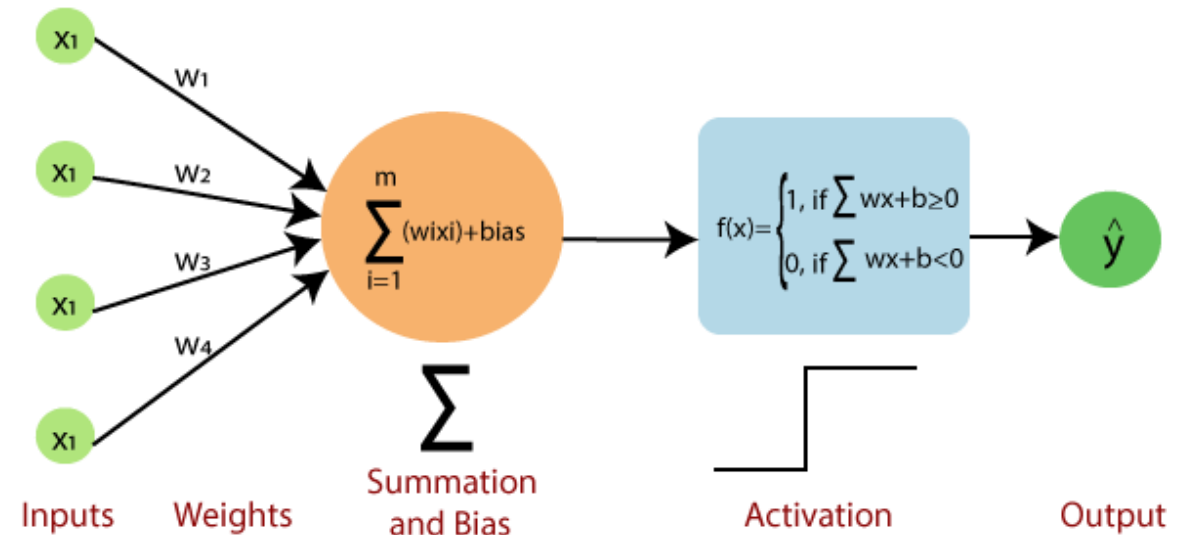
- Input: feature vector $x \in \mathbb{R}^m$
- Label: $y \in \{0, 1\}$
- Model:

$$\hat{y} = \sigma(w^T x + b)$$

- **Update rule:**

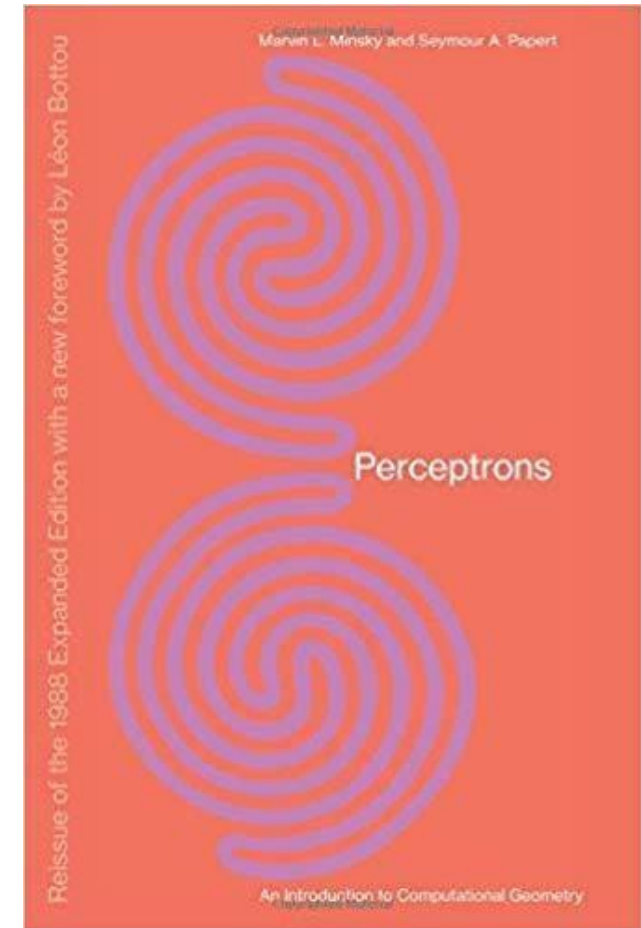
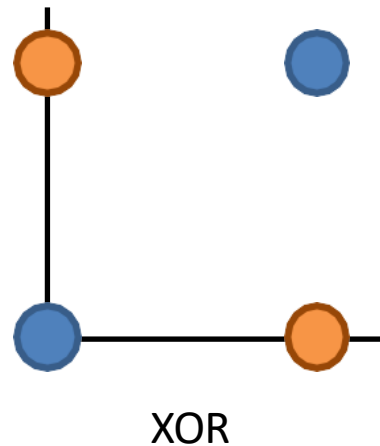
$$w_{t+1} = w_t + (y - \hat{y})x$$

label Prediction



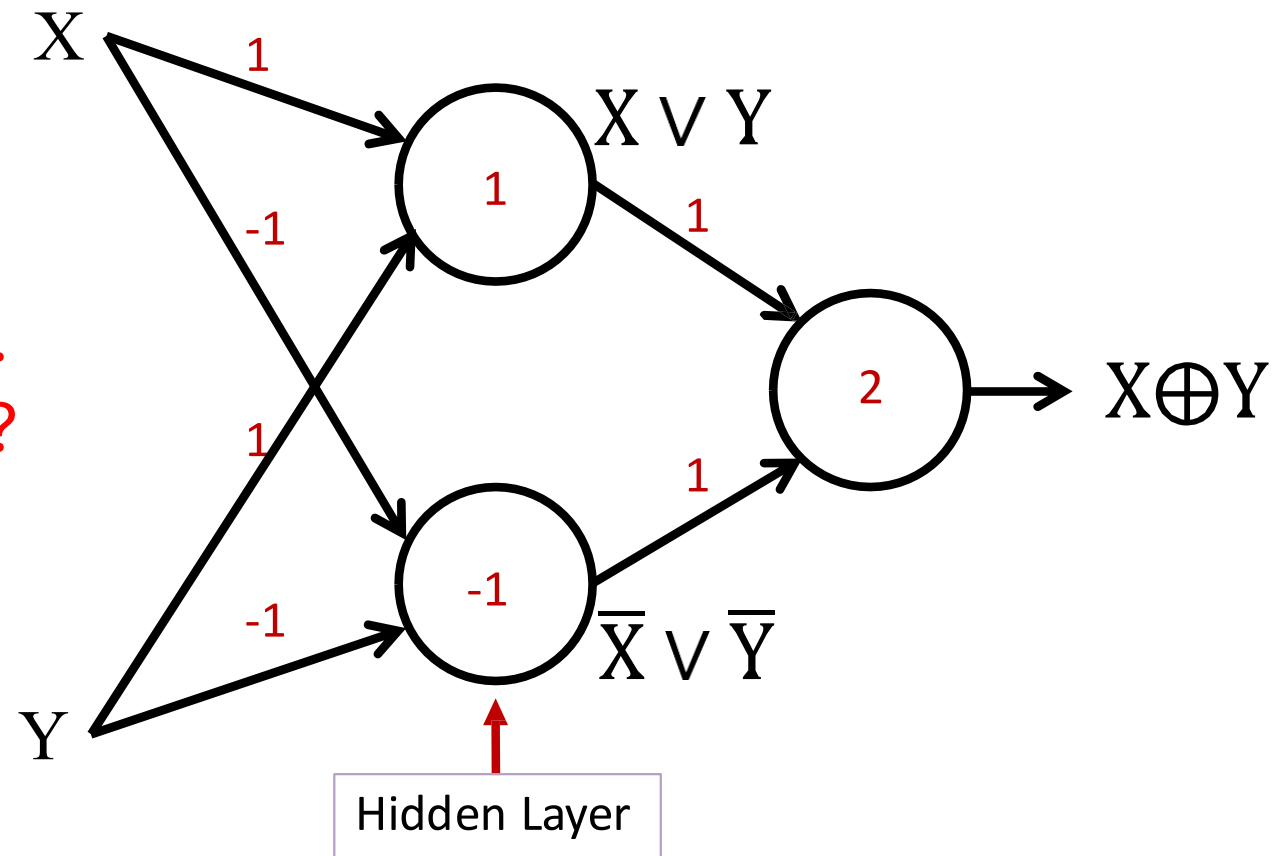
Minsky and Papert, 1969 (AI Winter)

- Showed that Perceptron could not learn the XOR function



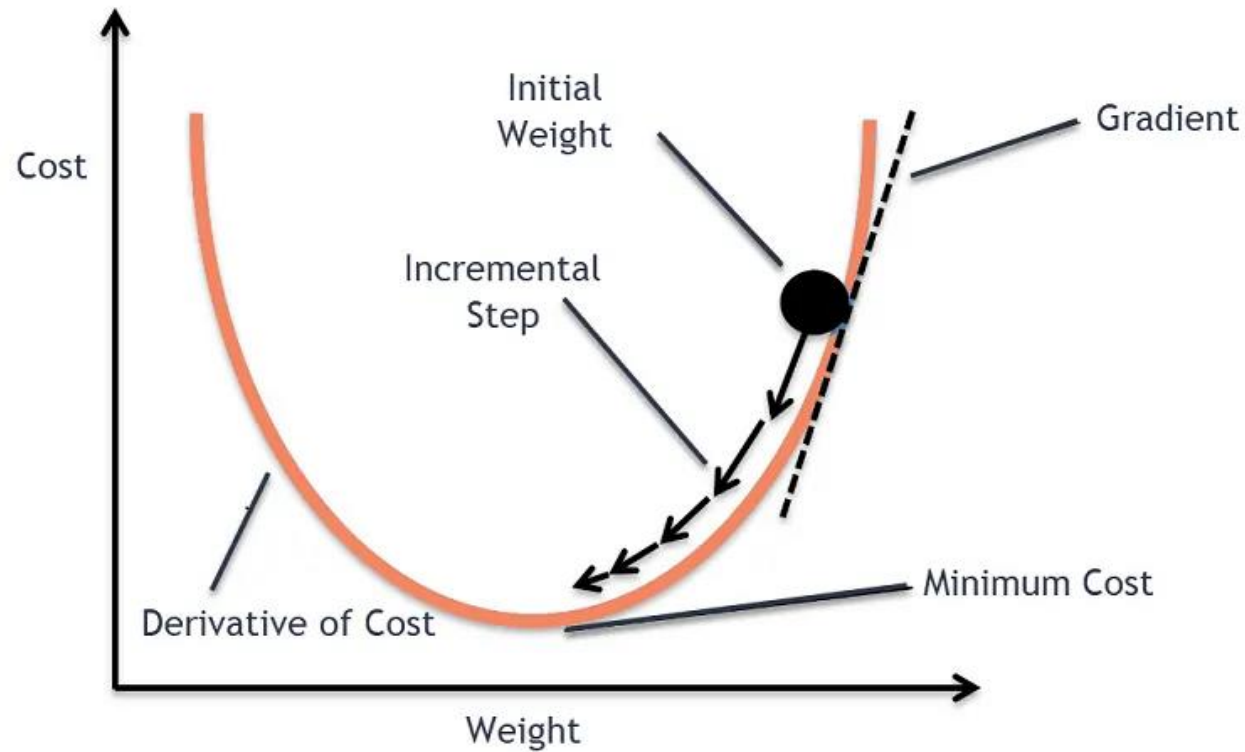
XOR and Multi-layer Perceptron (MLP)

Solution exists...
But how to train?

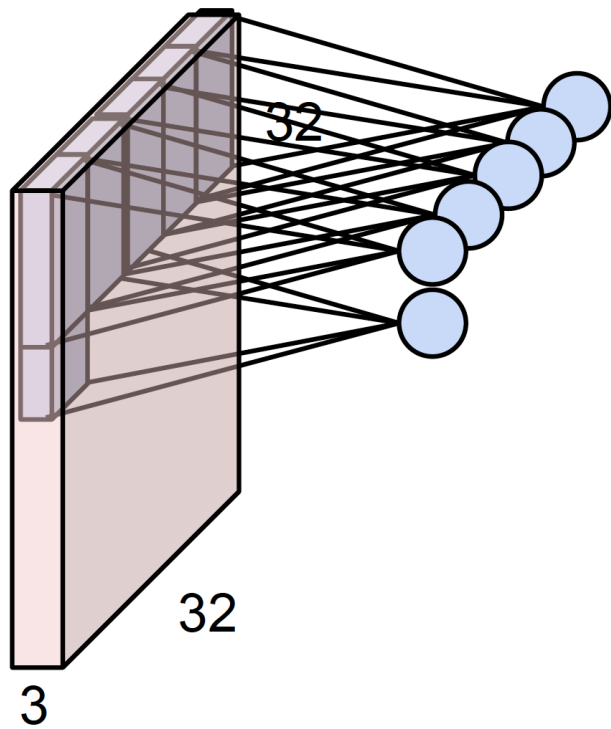


About training...

Cauchy and the Gradient Method (1857)



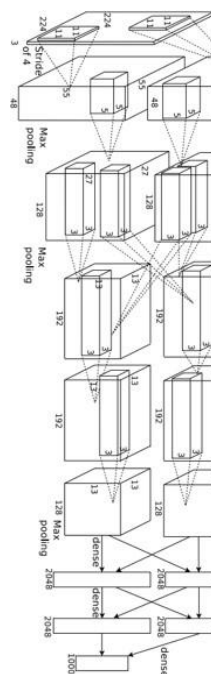
Convolutional Layer



Deep Conv Nets

Year 2012

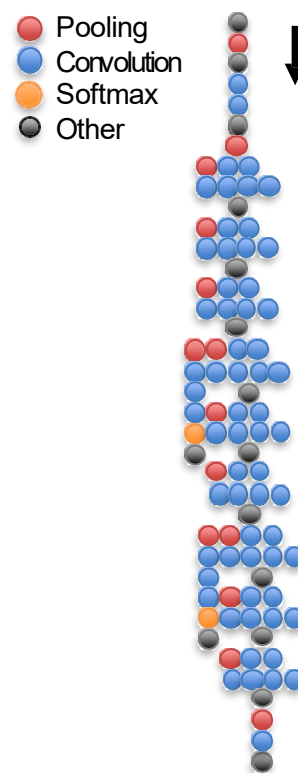
AlexNet



[Krizhevsky NIPS 2012]

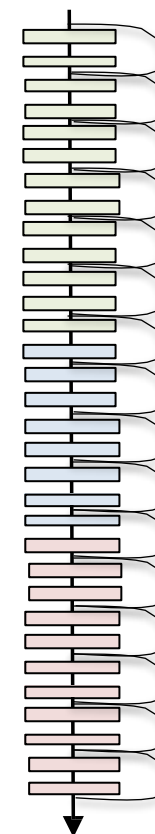
Year 2014

GoogLeNet

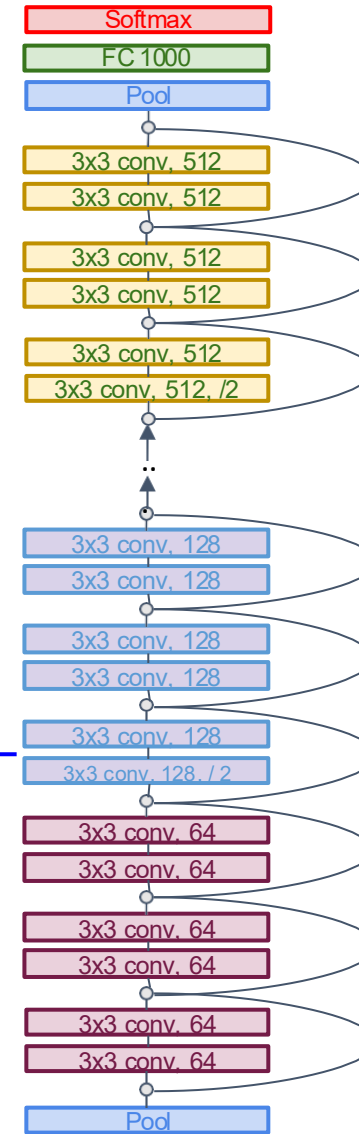
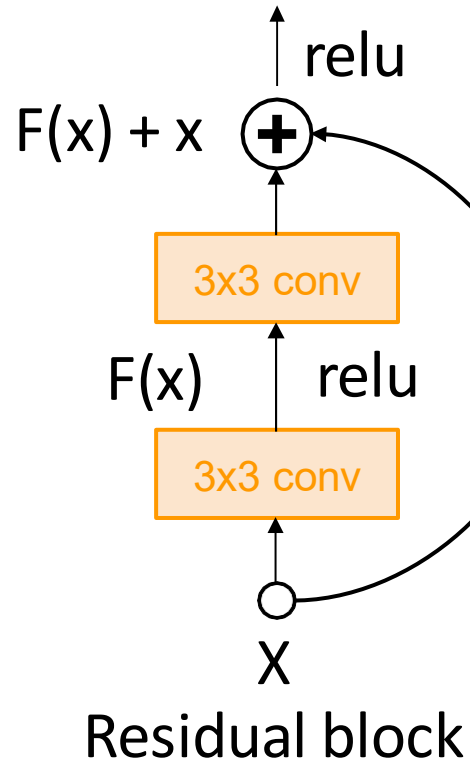


Year 2015

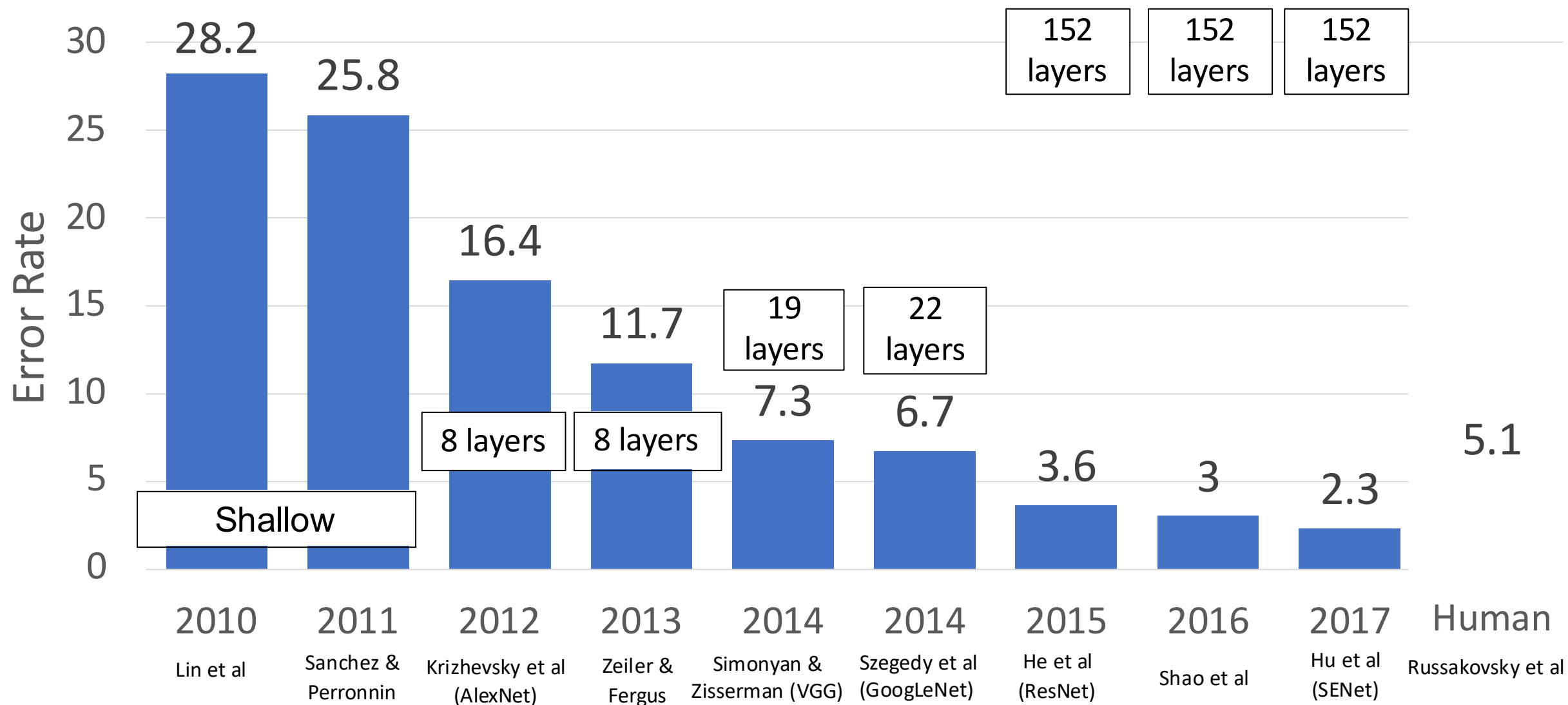
ResNet



ResNet (Skip Connections)



ImageNet Classification Challenge





So, we can solve image classification

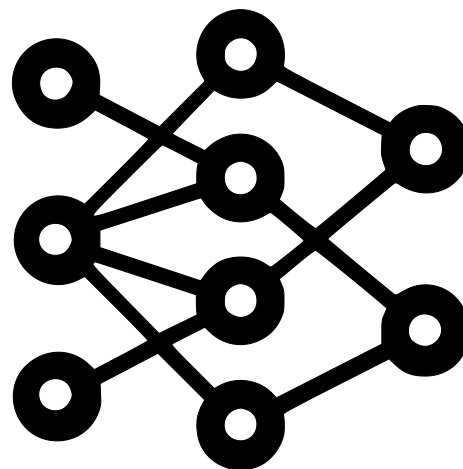
Real world input



Model
input

$$\begin{bmatrix} 124 \\ 140 \\ 156 \\ 128 \\ 142 \\ 157 \\ \vdots \end{bmatrix}$$

Model



Model
output

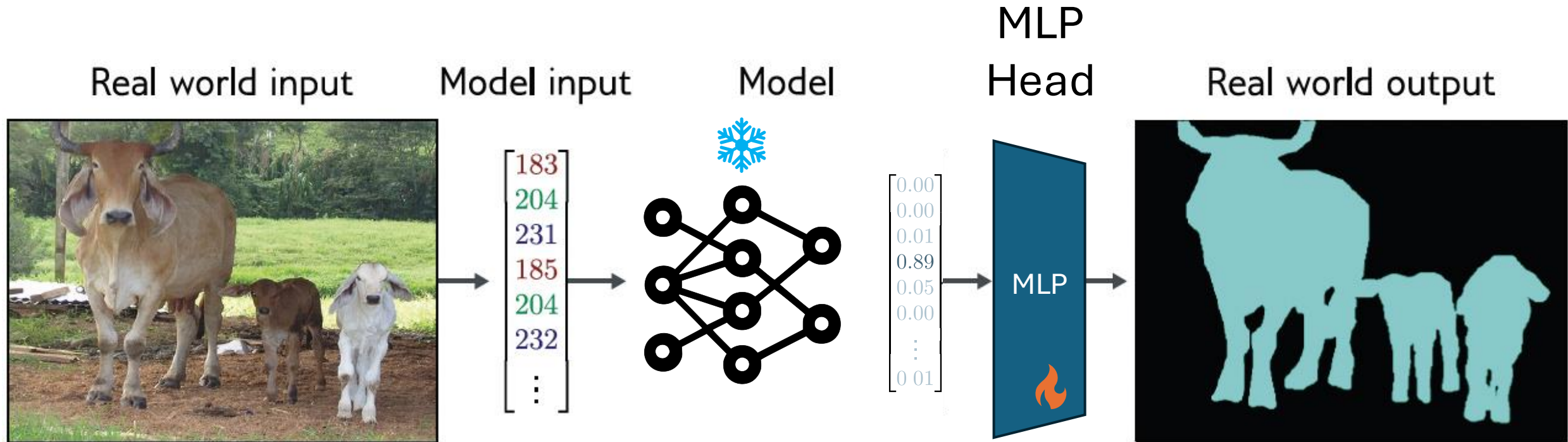
$$\begin{bmatrix} 0.00 \\ 0.00 \\ 0.01 \\ 0.89 \\ 0.05 \\ 0.00 \\ \vdots \\ 0.01 \end{bmatrix}$$

Real world output

Aardvark
Apple
Bee
Bicycle
Bridge
Clown
⋮

Classification is not why deep learning got famous

Transfer learning: Image segmentation



Transfer learning: Depth estimation

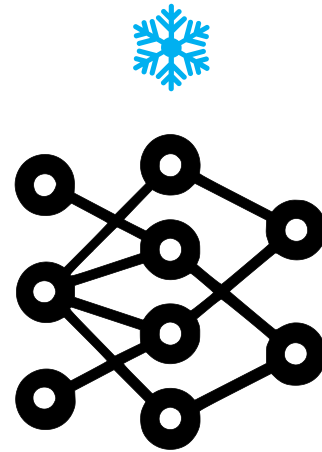
Real world input



Model
input

$\begin{bmatrix} 255 \\ 254 \\ 255 \\ 254 \\ 254 \\ 255 \\ \vdots \end{bmatrix}$

Model

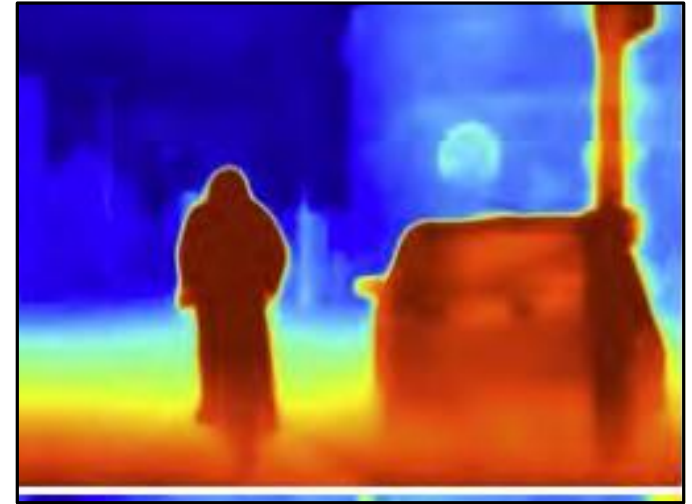


MLP
Head

$\begin{bmatrix} 0.00 \\ 0.00 \\ 0.01 \\ 0.89 \\ 0.05 \\ 0.00 \\ \vdots \\ 0.01 \end{bmatrix}$



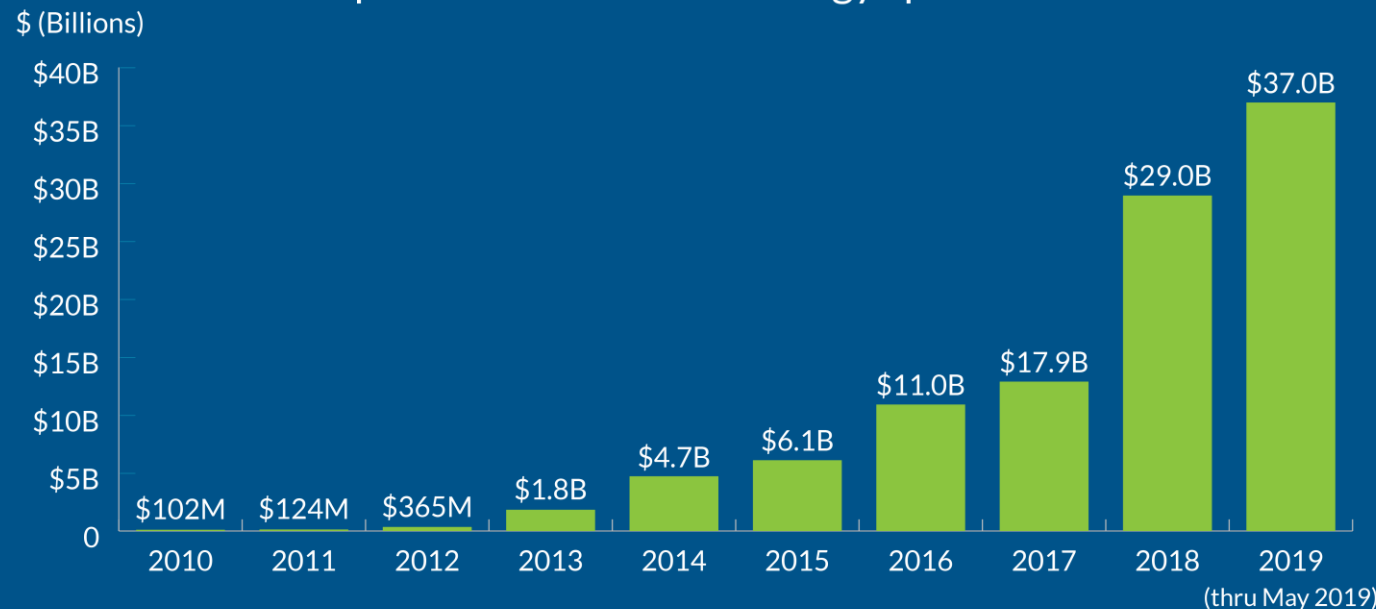
Real world output



Investment in autonomous vehicles

Never before in the automotive industry have we seen this level of investment in new technologies...

Total *cumulative* funding (USD) for companies in the AV technology space



...and AV investments are poised to set a new record in 2019!



Think like a robot, perceive like a human.

*Source: Crunchbase

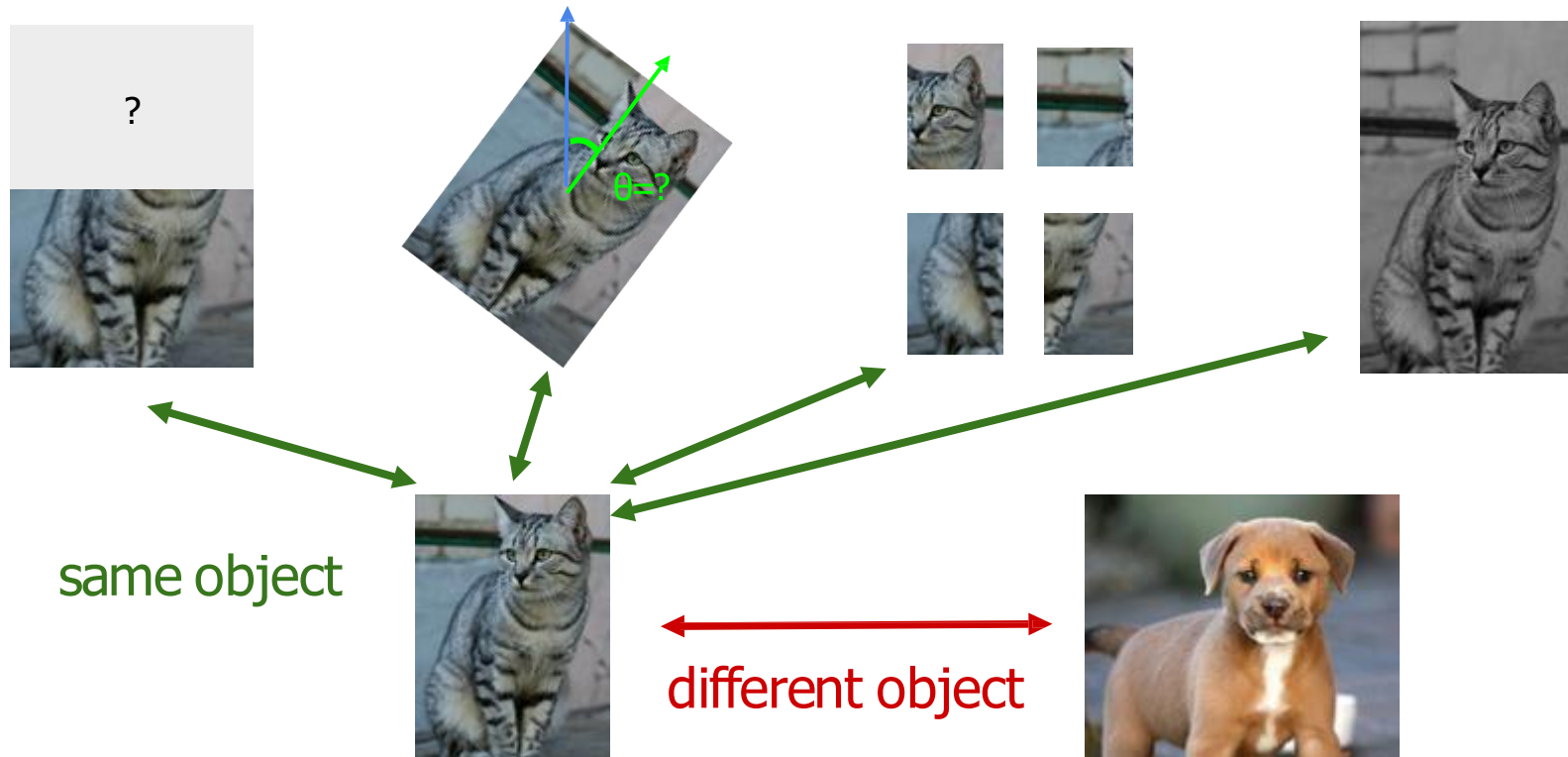
Representation learning, challenges

- ResNet-50 with ImageNet supervision

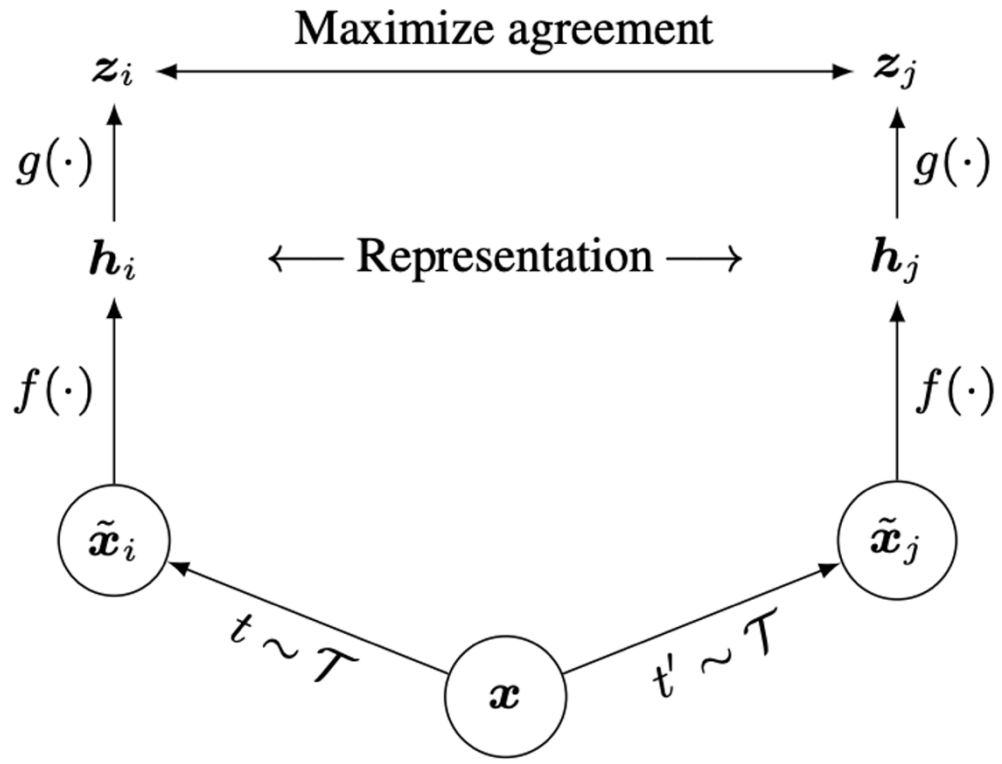
Labels Fraction	1%	10%	100%
Top1 Acc	25.4%	56.4%	78.9%
Top-5 Acc	48.4%	80.4%	94.2%

- 10% training data = 100,000 samples
- High workload for annotations
- We can't rely on supervision

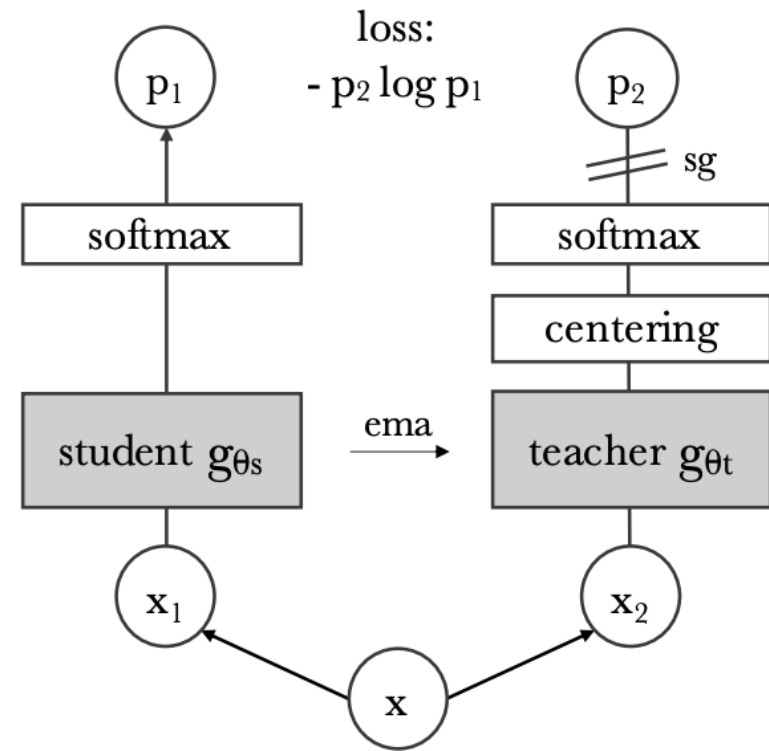
We need more data for better representation: Self-supervised models



Self-supervised Models

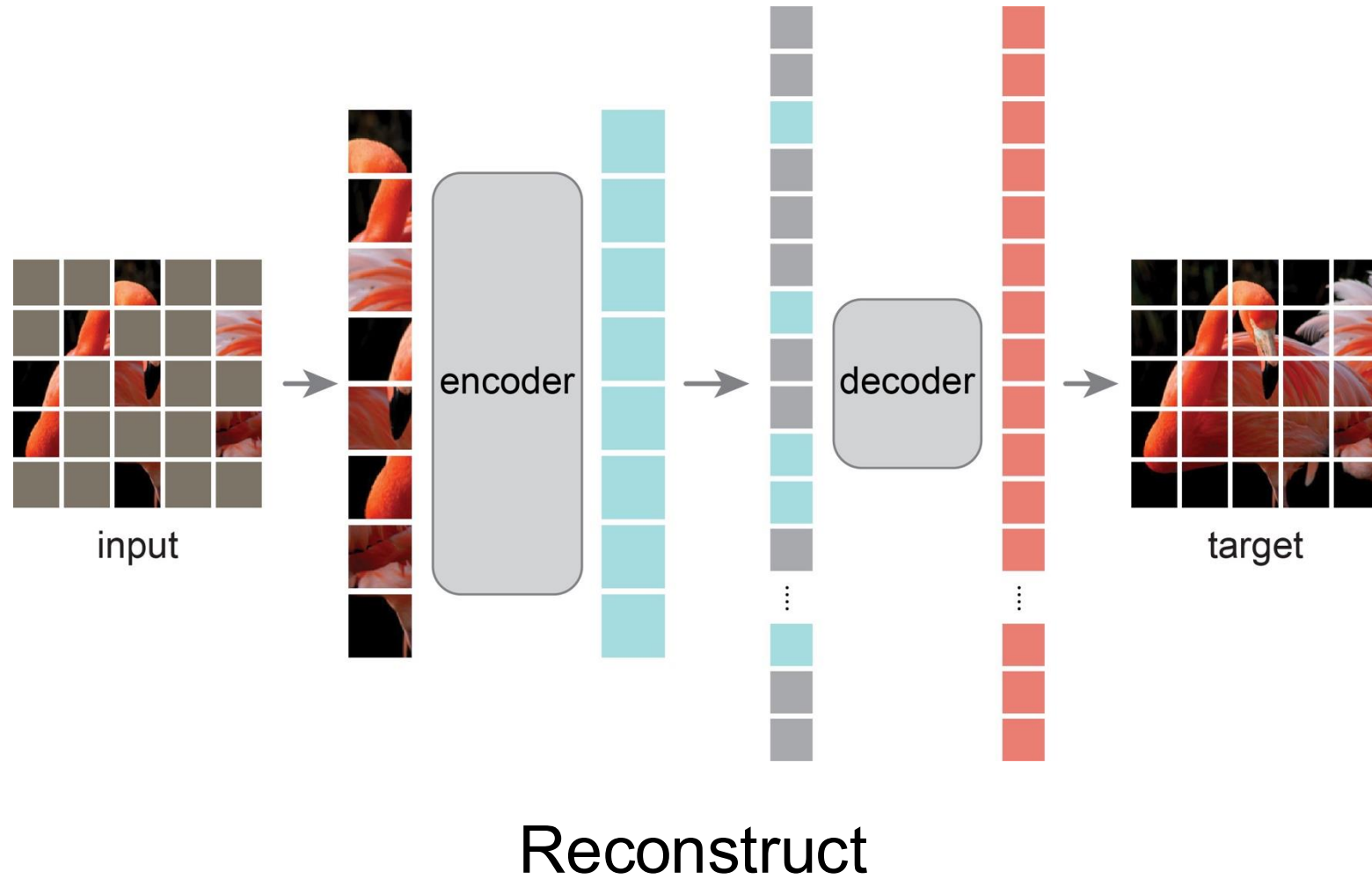


SimCLR

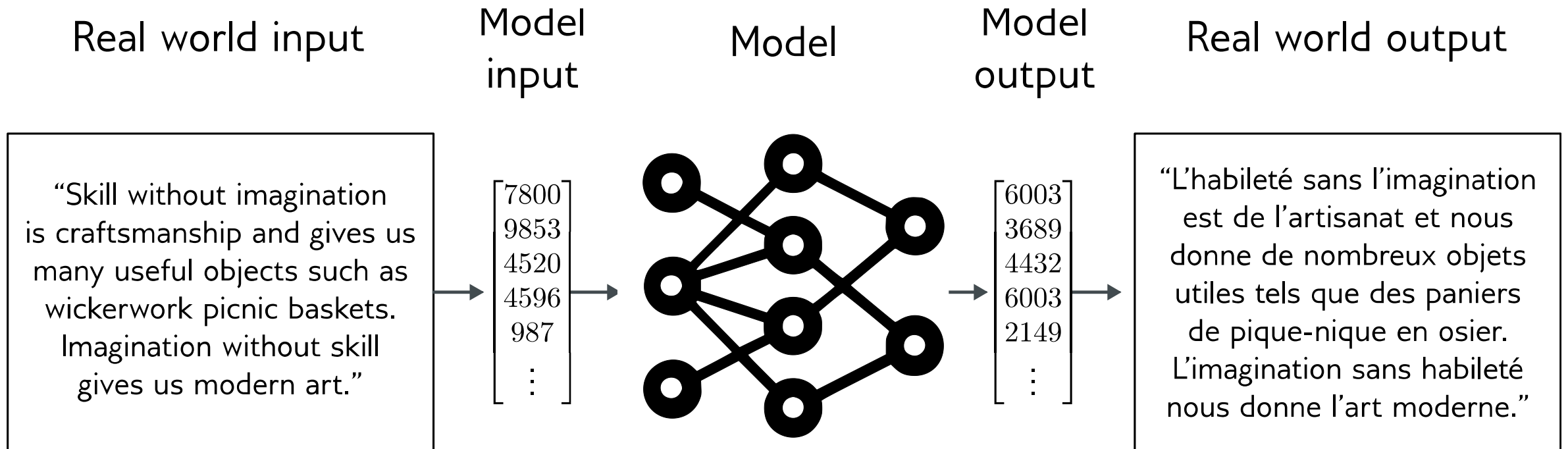


DINO

Reconstruction objective



What about text?



Sequence-to-Sequence with RNNs

Input: Sequence $x_1, \dots, x_T$

Output: Sequence $y_1, \dots, y_{T'}$

Decoder: $s_t = g_U(y_{t-1}, h_{t-1}, c)$

Encoder: $h_t = f_W(x_t, h_{t-1})$

From final hidden state predict:

Initial decoder state s_0

Context vector c (often $c=h_T$)

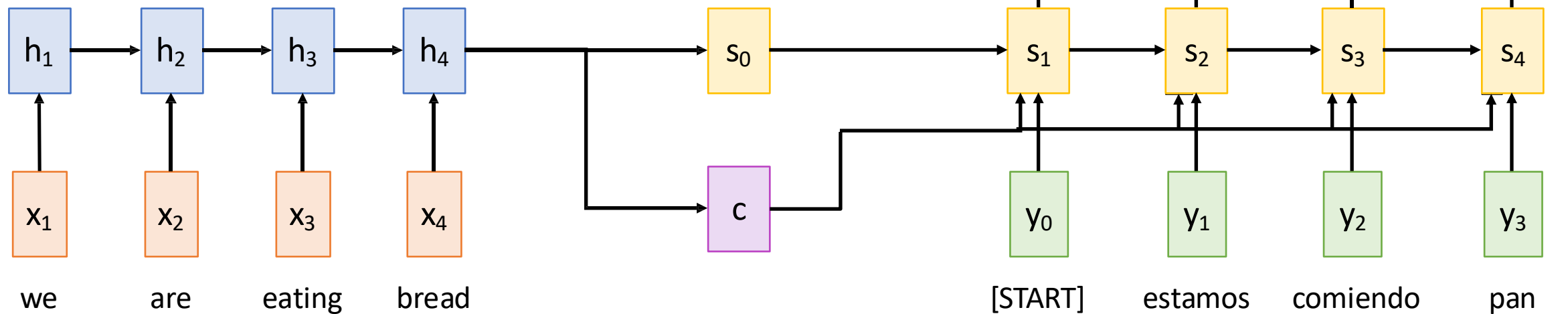


Image Captioning with RNNs and Attention

$$e_{t,i,j} = f_{\text{att}}(s_{t-1}, h_{i,j})$$
$$a_{t,:,:) = \text{softmax}(e_{t,:,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} h_{i,j}$$

Each timestep of decoder
uses a different context
vector that looks at different
parts of the input image

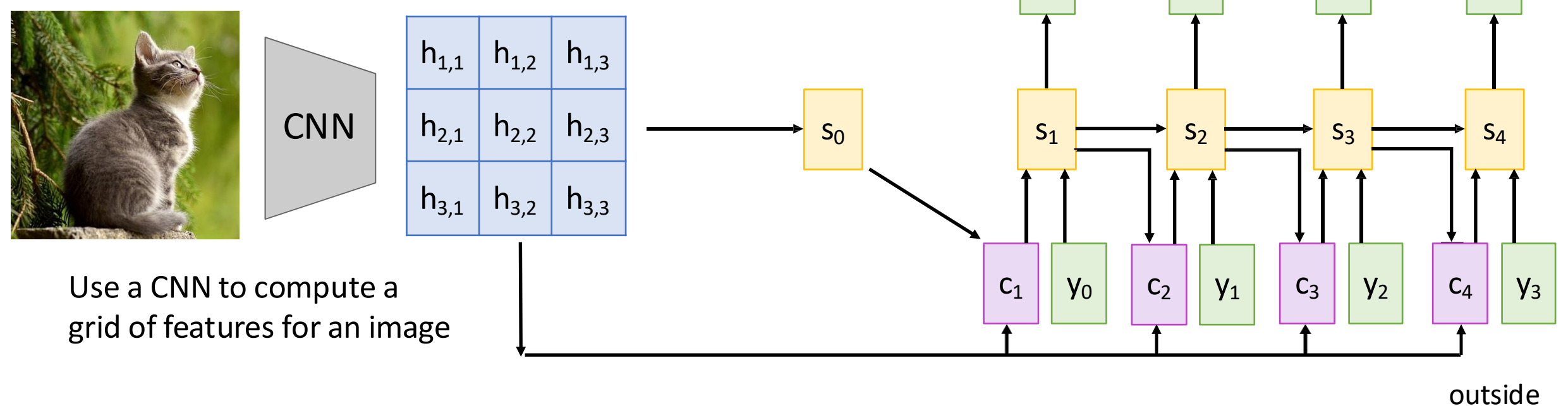
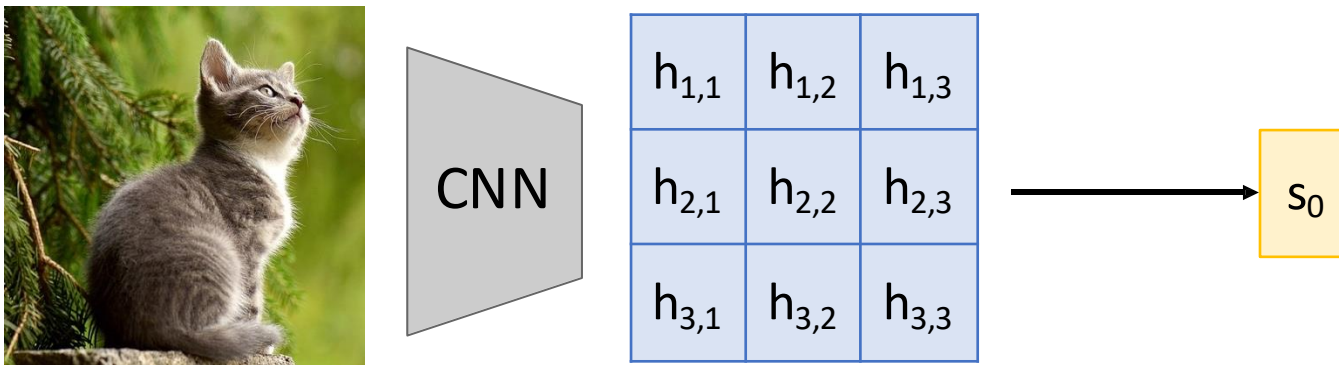


Image Captioning with RNNs and Attention



Use a CNN to compute a
grid of features for an image

Image Captioning with RNNs and Attention

$$e_{t,i,j} = f_{\text{att}}(s_{t-1}, h_{i,j})$$

Alignment scores

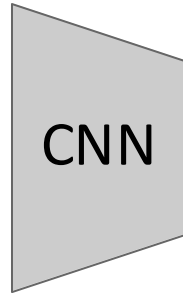
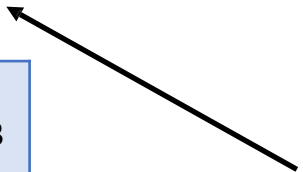
$e_{1,1,1}$	$e_{1,1,2}$	$e_{1,1,3}$
$e_{1,2,1}$	$e_{1,2,2}$	$e_{1,2,3}$
$e_{1,3,1}$	$e_{1,3,2}$	$e_{1,3,3}$



$h_{1,1}$	$h_{1,2}$	$h_{1,3}$
$h_{2,1}$	$h_{2,2}$	$h_{2,3}$
$h_{3,1}$	$h_{3,2}$	$h_{3,3}$



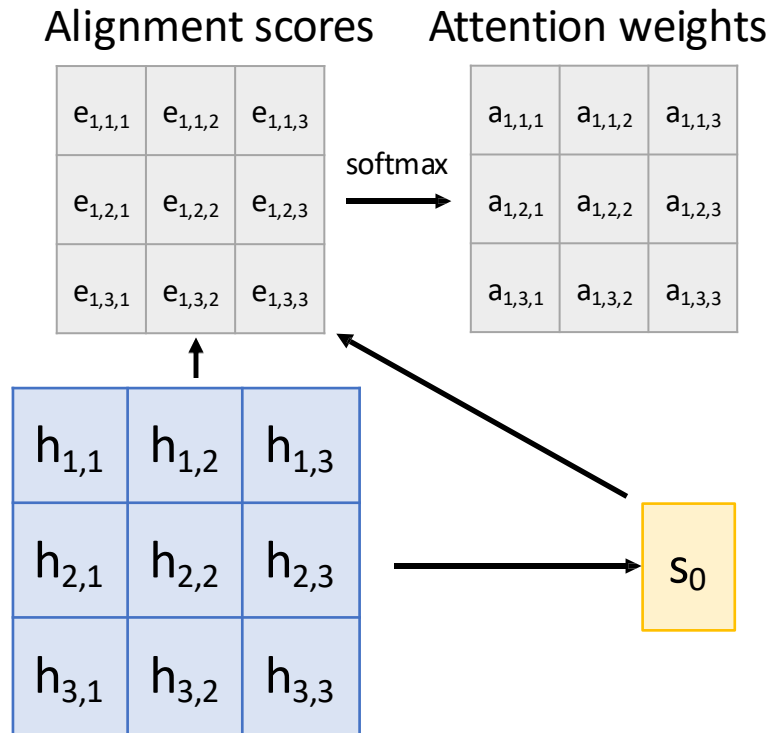
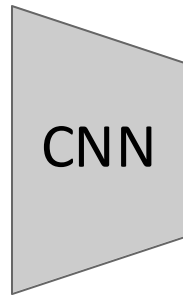
S_0



Use a CNN to compute a
grid of features for an image

Image Captioning with RNNs and Attention

$$e_{t,i,j} = f_{\text{att}}(s_{t-1}, h_{i,j})$$
$$a_{t,:} = \text{softmax}(e_{t,:,:})$$

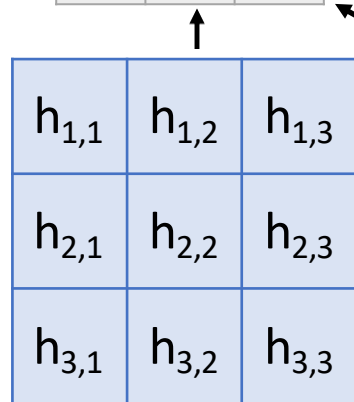
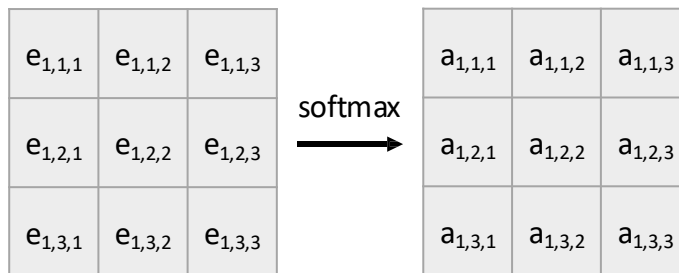


Use a CNN to compute a
grid of features for an image

Image Captioning with RNNs and Attention

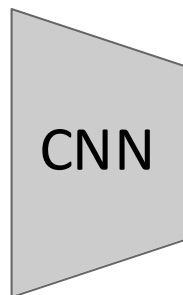
$$e_{t,i,j} = f_{\text{att}}(s_{t-1}, h_{i,j})$$
$$a_{t,:,:} = \text{softmax}(e_{t,:,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} h_{i,j}$$

Alignment scores Attention weights



s_0

c_1



Use a CNN to compute a grid of features for an image

Image Captioning with RNNs and Attention

$$e_{t,i,j} = f_{\text{att}}(s_{t-1}, h_{i,j})$$

$$a_{t,:,:) = \text{softmax}(e_{t,:,:})$$

$$c_t = \sum_{i,j} a_{t,i,j} h_{i,j}$$



Use a CNN to compute a grid of features for an image

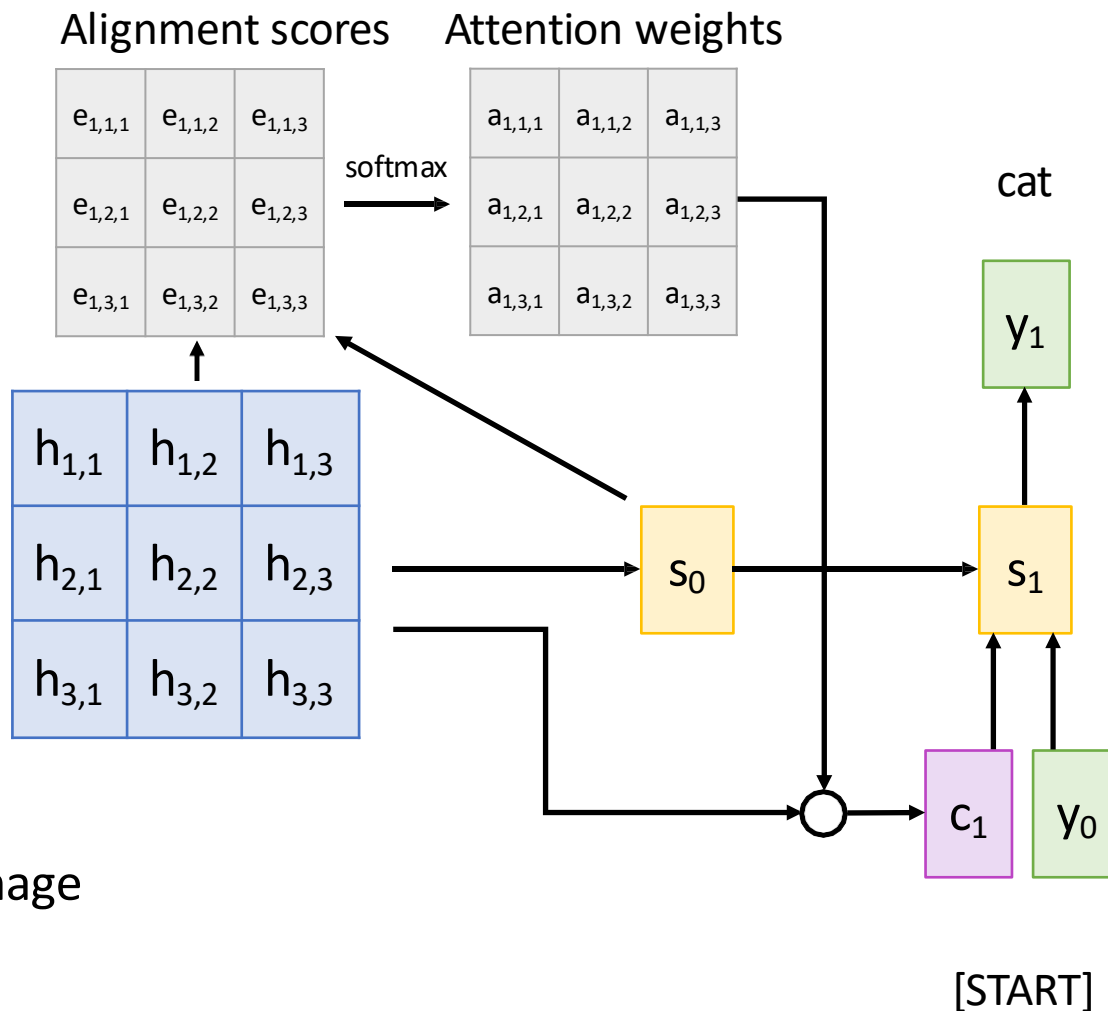


Image Captioning with RNNs and Attention

$$e_{t,i,j} = f_{\text{att}}(s_{t-1}, h_{i,j})$$
$$a_{t,:} = \text{softmax}(e_{t,:,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} h_{i,j}$$

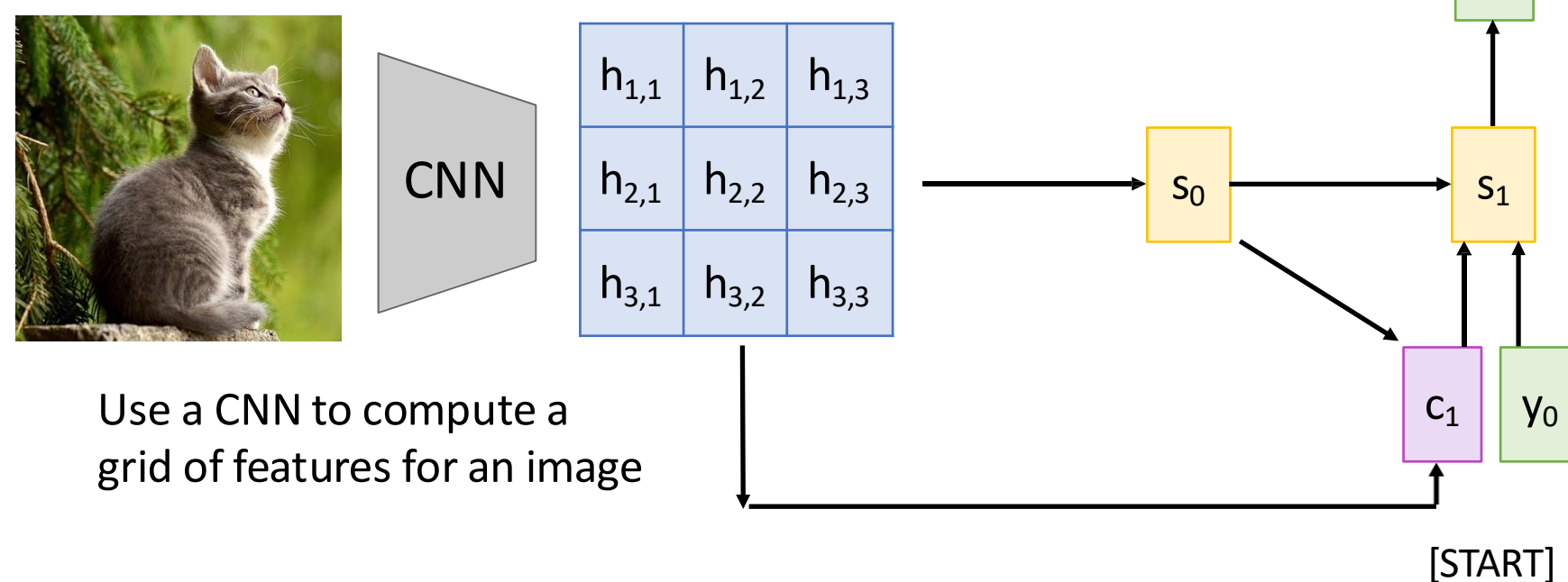


Image Captioning with RNNs and Attention

$$e_{t,i,j} = f_{\text{att}}(s_{t-1}, h_{i,j})$$
$$a_{t,:,:} = \text{softmax}(e_{t,:,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} h_{i,j}$$

Alignment scores

$e_{2,1,1}$	$e_{2,1,2}$	$e_{2,1,3}$
$e_{2,2,1}$	$e_{2,2,2}$	$e_{2,2,3}$
$e_{2,3,1}$	$e_{2,3,2}$	$e_{2,3,3}$



$h_{1,1}$	$h_{1,2}$	$h_{1,3}$
$h_{2,1}$	$h_{2,2}$	$h_{2,3}$
$h_{3,1}$	$h_{3,2}$	$h_{3,3}$

cat

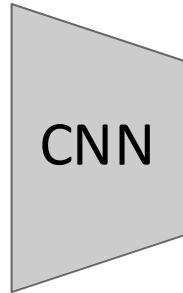
y_1

s_1

c_1

y_0

[START]



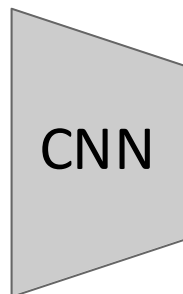
Use a CNN to compute a grid of features for an image

Image Captioning with RNNs and Attention

$$e_{t,i,j} = f_{\text{att}}(s_{t-1}, h_{i,j})$$
$$a_{t,:,:} = \text{softmax}(e_{t,:,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} h_{i,j}$$

Alignment scores Attention weights

$e_{2,1,1}$	$e_{2,1,2}$	$e_{2,1,3}$	softmax →	$a_{2,1,1}$	$a_{2,1,2}$	$a_{2,1,3}$
$e_{2,2,1}$	$e_{2,2,2}$	$e_{2,2,3}$		$a_{2,2,1}$	$a_{2,2,2}$	$a_{2,2,3}$
$e_{2,3,1}$	$e_{2,3,2}$	$e_{2,3,3}$		$a_{2,3,1}$	$a_{2,3,2}$	$a_{2,3,3}$



Use a CNN to compute a
grid of features for an image

$h_{1,1}$	$h_{1,2}$	$h_{1,3}$
$h_{2,1}$	$h_{2,2}$	$h_{2,3}$
$h_{3,1}$	$h_{3,2}$	$h_{3,3}$

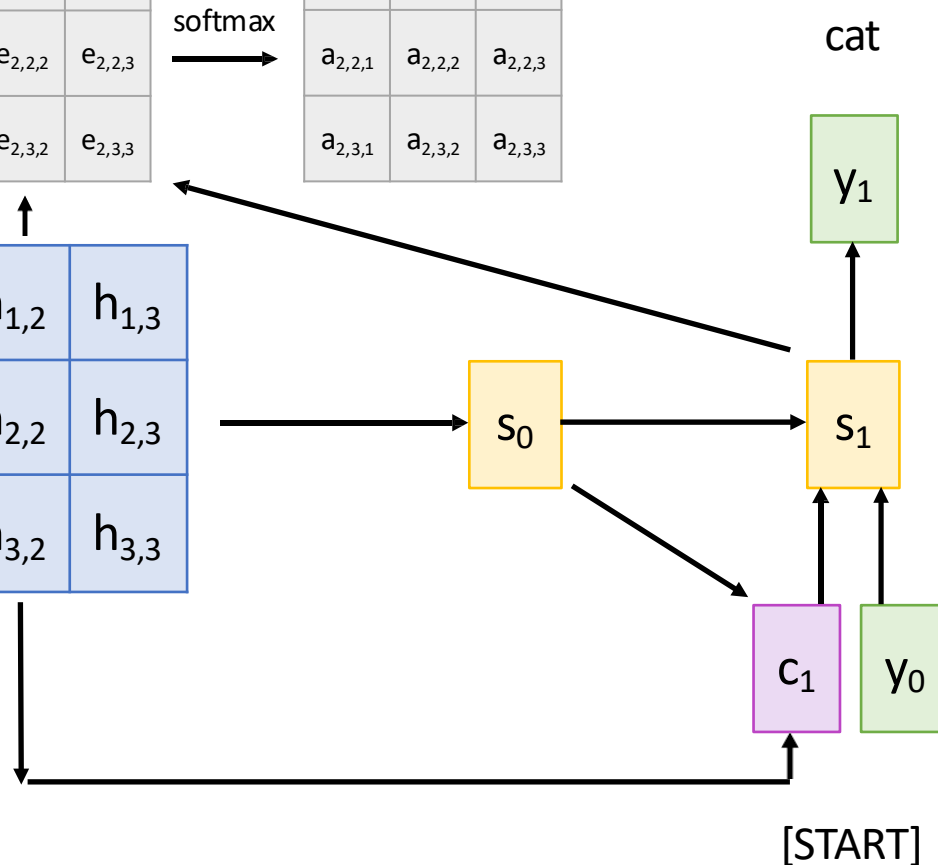
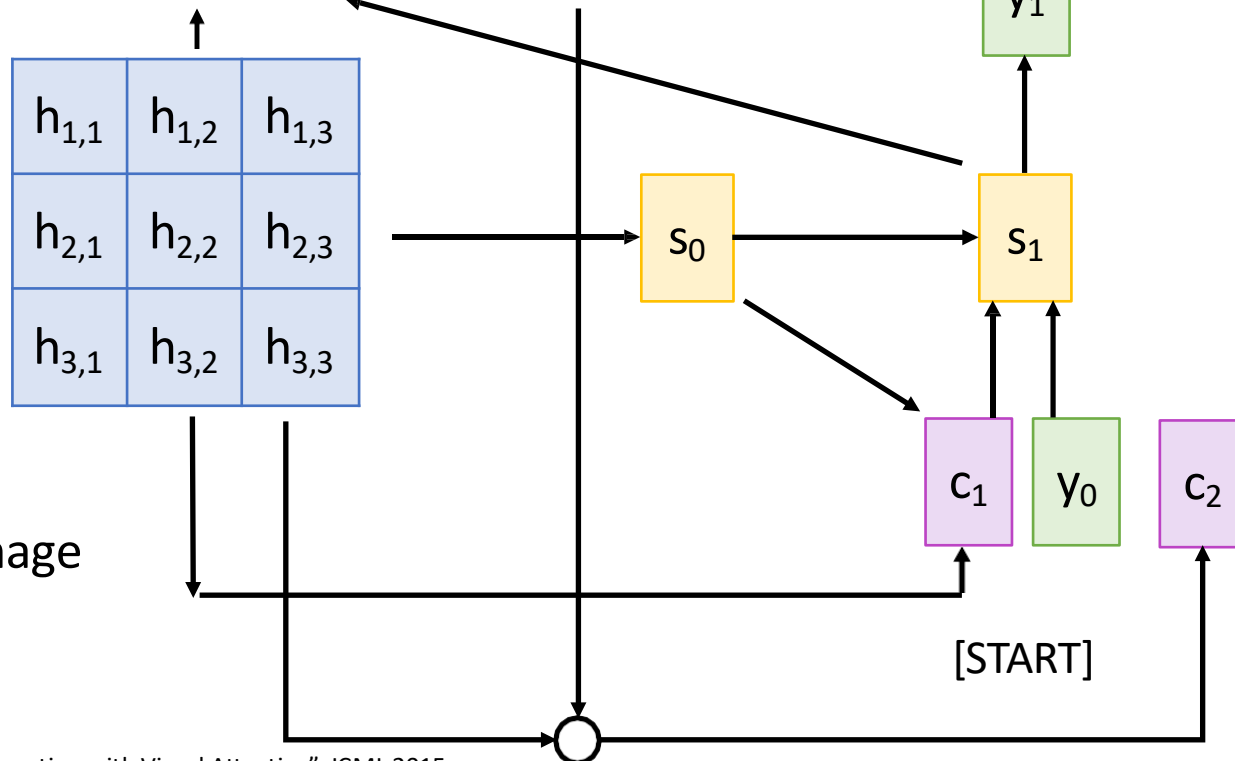


Image Captioning with RNNs and Attention

$$e_{t,i,j} = f_{\text{att}}(s_{t-1}, h_{i,j})$$
$$a_{t,:,:} = \text{softmax}(e_{t,:,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} h_{i,j}$$

Alignment scores Attention weights

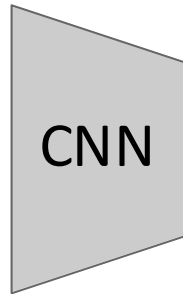
$e_{2,1,1}$	$e_{2,1,2}$	$e_{2,1,3}$	softmax →	$a_{2,1,1}$	$a_{2,1,2}$	$a_{2,1,3}$
$e_{2,2,1}$	$e_{2,2,2}$	$e_{2,2,3}$		$a_{2,2,1}$	$a_{2,2,2}$	$a_{2,2,3}$
$e_{2,3,1}$	$e_{2,3,2}$	$e_{2,3,3}$		$a_{2,3,1}$	$a_{2,3,2}$	$a_{2,3,3}$



Use a CNN to compute a grid of features for an image

Image Captioning with RNNs and Attention

$$e_{t,i,j} = f_{\text{att}}(s_{t-1}, h_{i,j})$$
$$a_{t,:,:) = \text{softmax}(e_{t,:,:})$$
$$c_t = \sum_{i,j} a_{t,i,j} h_{i,j}$$



Use a CNN to compute a grid of features for an image

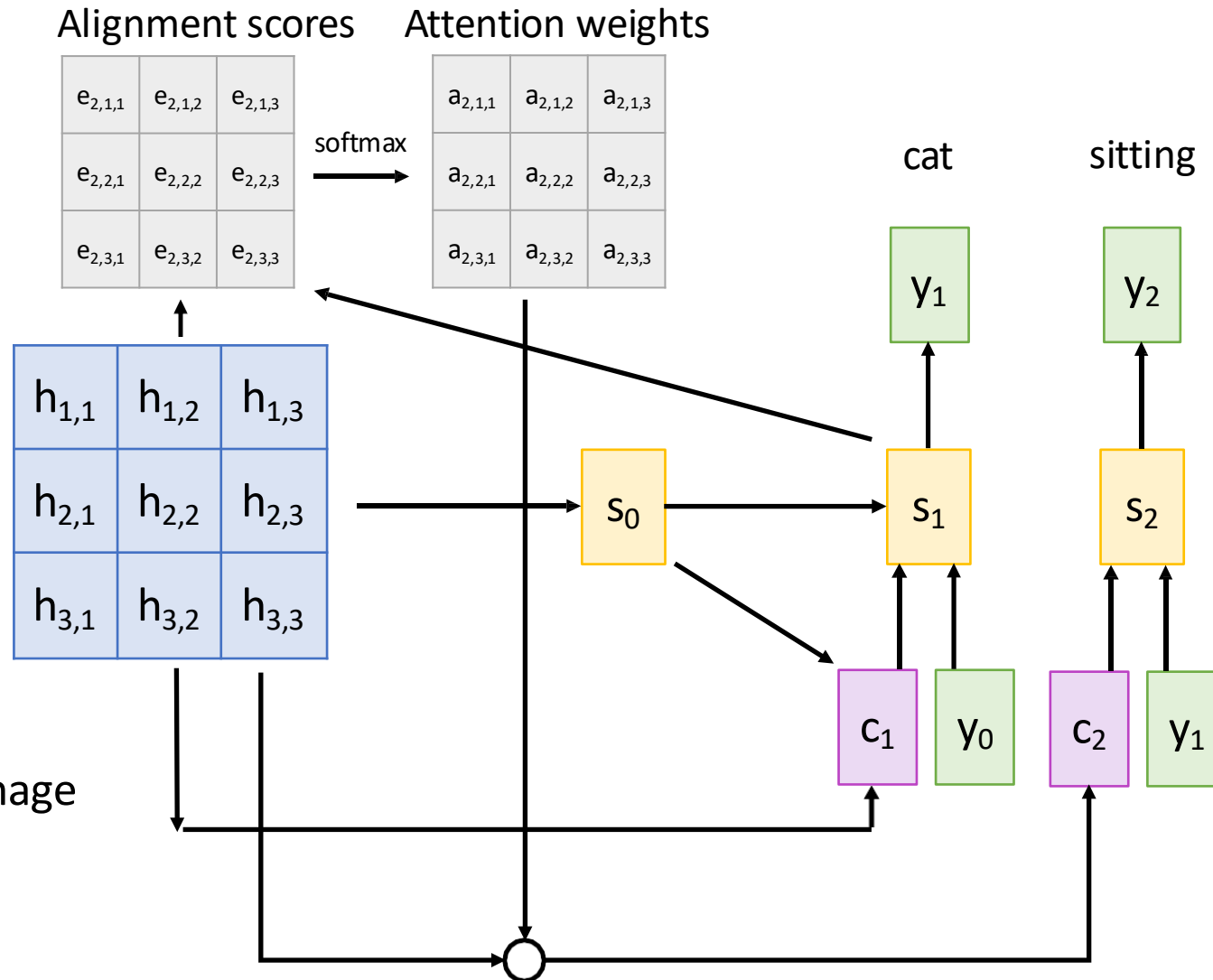
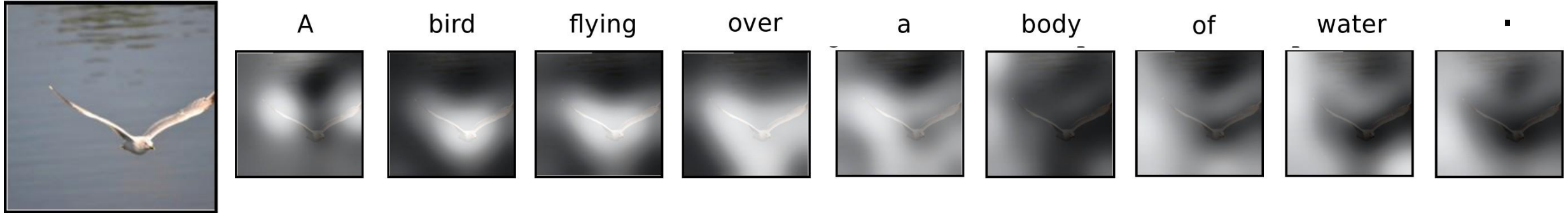


Image Captioning with RNNs and Attention



X, Attend, and Y

“Show, attend, and tell” (*Xu et al, ICML 2015*)

Look at image, attend to image regions, produce question

“Ask, attend, and answer” (*Xu and Saenko, ECCV 2016*)

“Show, ask, attend, and answer” (*Kazemi and Elqursh, 2017*)

Read text of question, attend to image regions, produce answer

“Listen, attend, and spell” (*Chan et al, ICASSP 2016*)

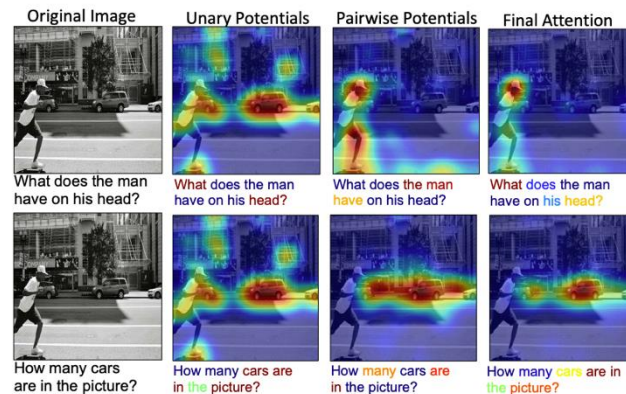
Process raw audio, attend to audio regions while producing text

“Listen, attend, and walk” (*Mei et al, AAAI 2016*)

Process text, attend to text regions, output navigation commands

Multimodal **Attention** – the bigger the input the more crucial attention is

Visual Question Answering [1]



Visual Dialog [2]

History

Q: how many kids ?
A: 5
Q: is this birthday party?
A: yes
Q: any adults ?
A: lady lighting candle
Q: is this outdoors ?
A: not very sure
Q: is table round ?
A: no



are kids wearing hats ?

Video Dialog [3]

Question: where is the video taking place ?

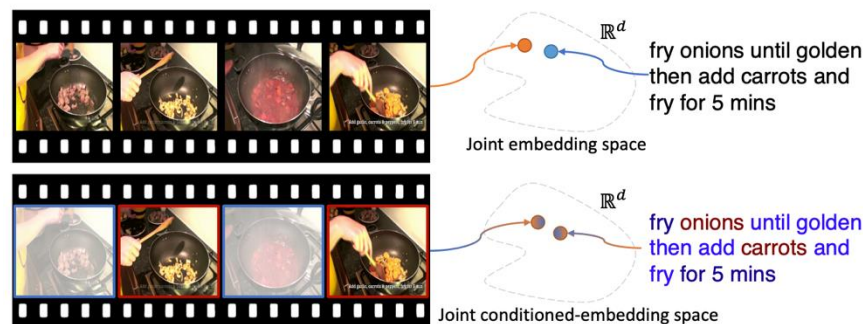


MultiModal-Attention:

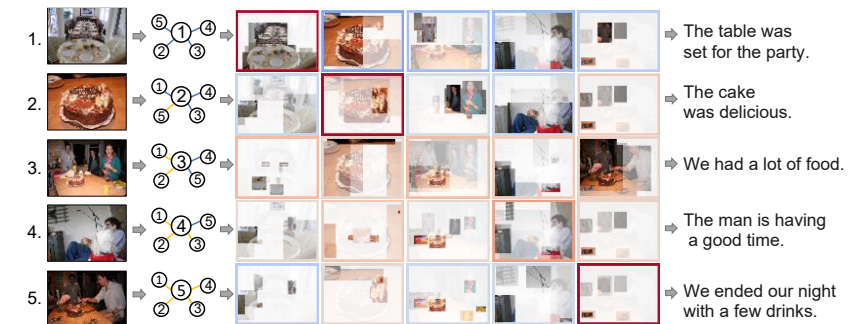


Answer: the video starts with a man in the kitchen .

Video retrieval [4]



Visual storytelling [5]



[1] - High-order attention models for visual question answering [Schwartz et al.; NIPS'17]

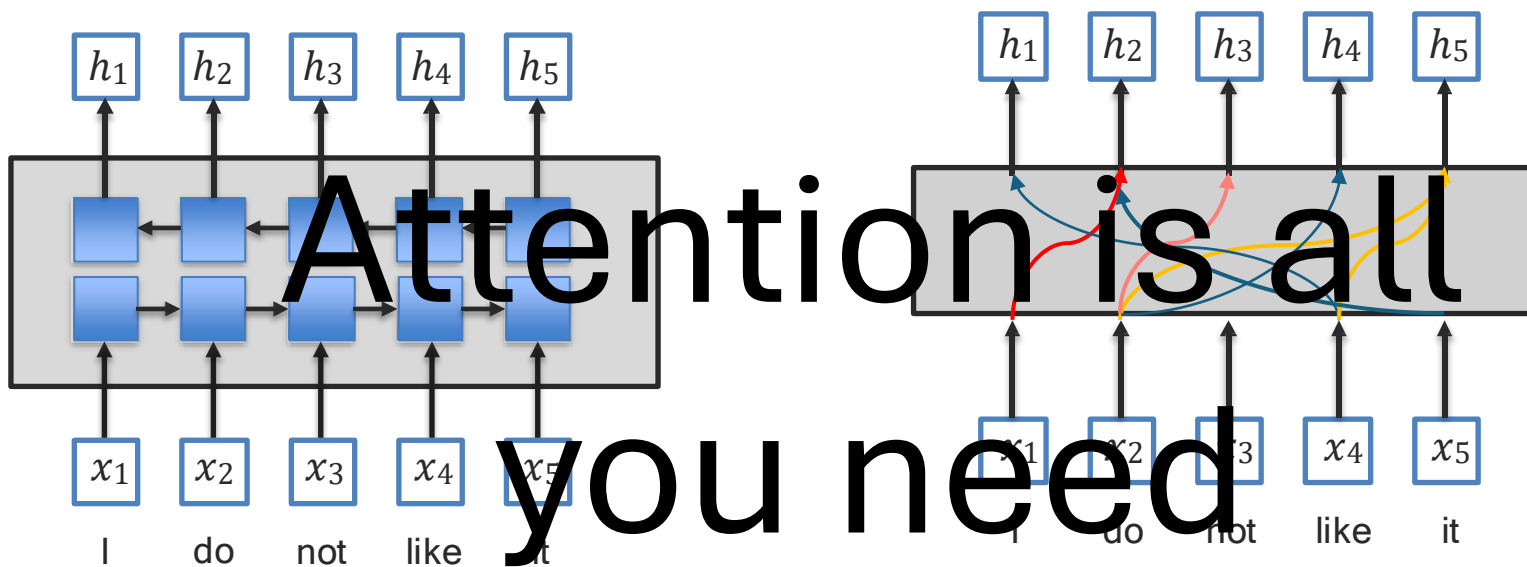
[2] - Factor Graph Attention [Schwartz et al.; CVPR'19]

[3] - A simple baseline for audio-visual scene-aware dialog [Schwartz et al. CVPR'19]

[4] - Video and Text Matching with Conditioned Embeddings [Ali et al. WACV'22]

[5] - Ordered attention for coherent visual storytelling [Braude et al. ACM-MM'22]

Attention for sequence representation



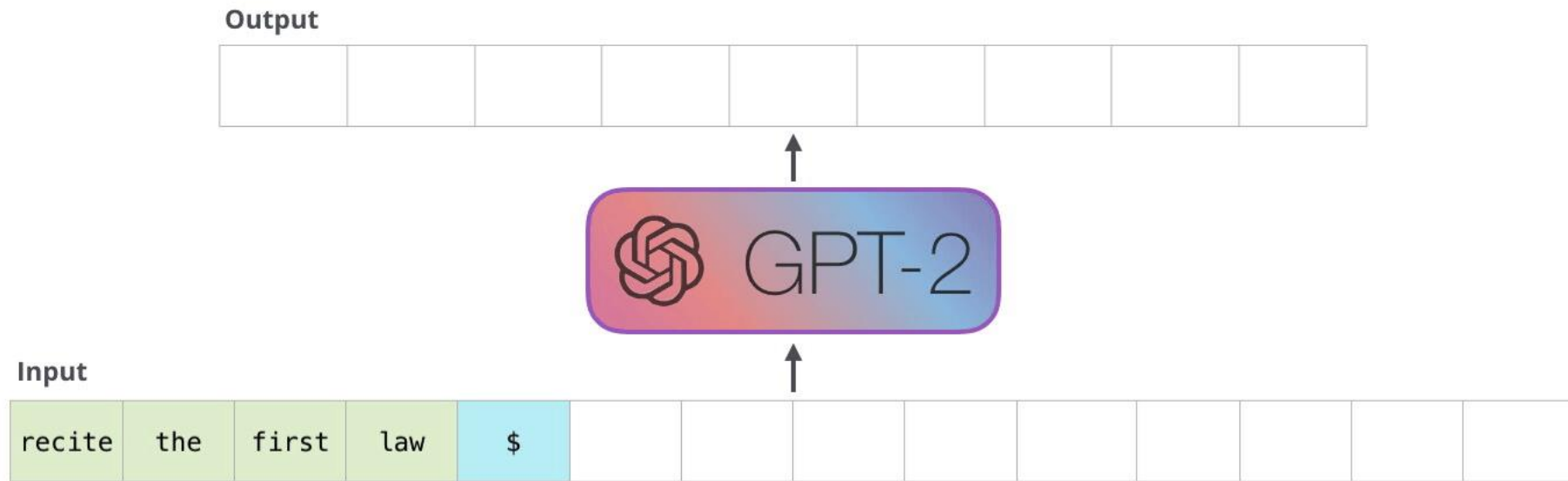
Can process long sequence

But harder to parallelize...

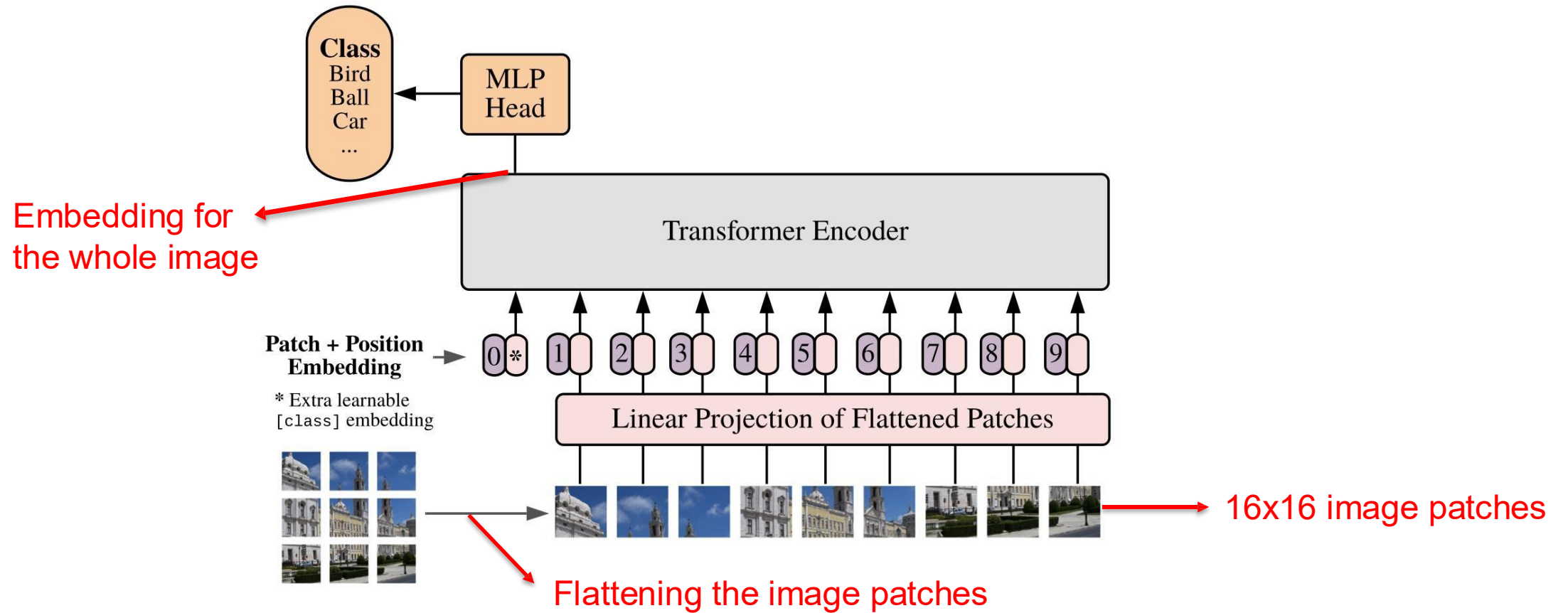
Can be parallelized!

Memory expensive

GPT: next-word prediction is all you need

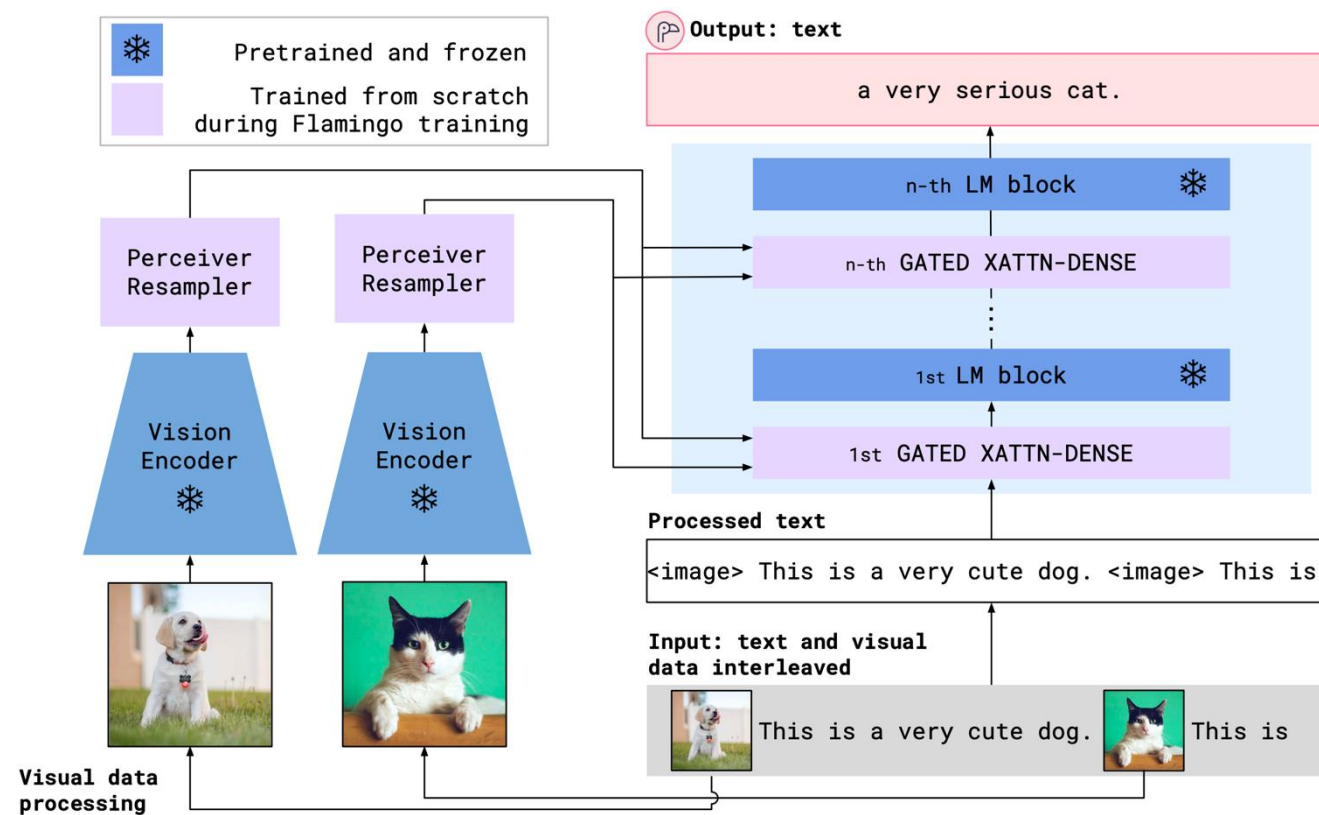


Vision Transformer



Flamingo

- Employ 70B Frozen LLM
- Large-scale dataset: 43M web pages, Multimodal MassiveWeb (M3W)



Scaling up Transformers

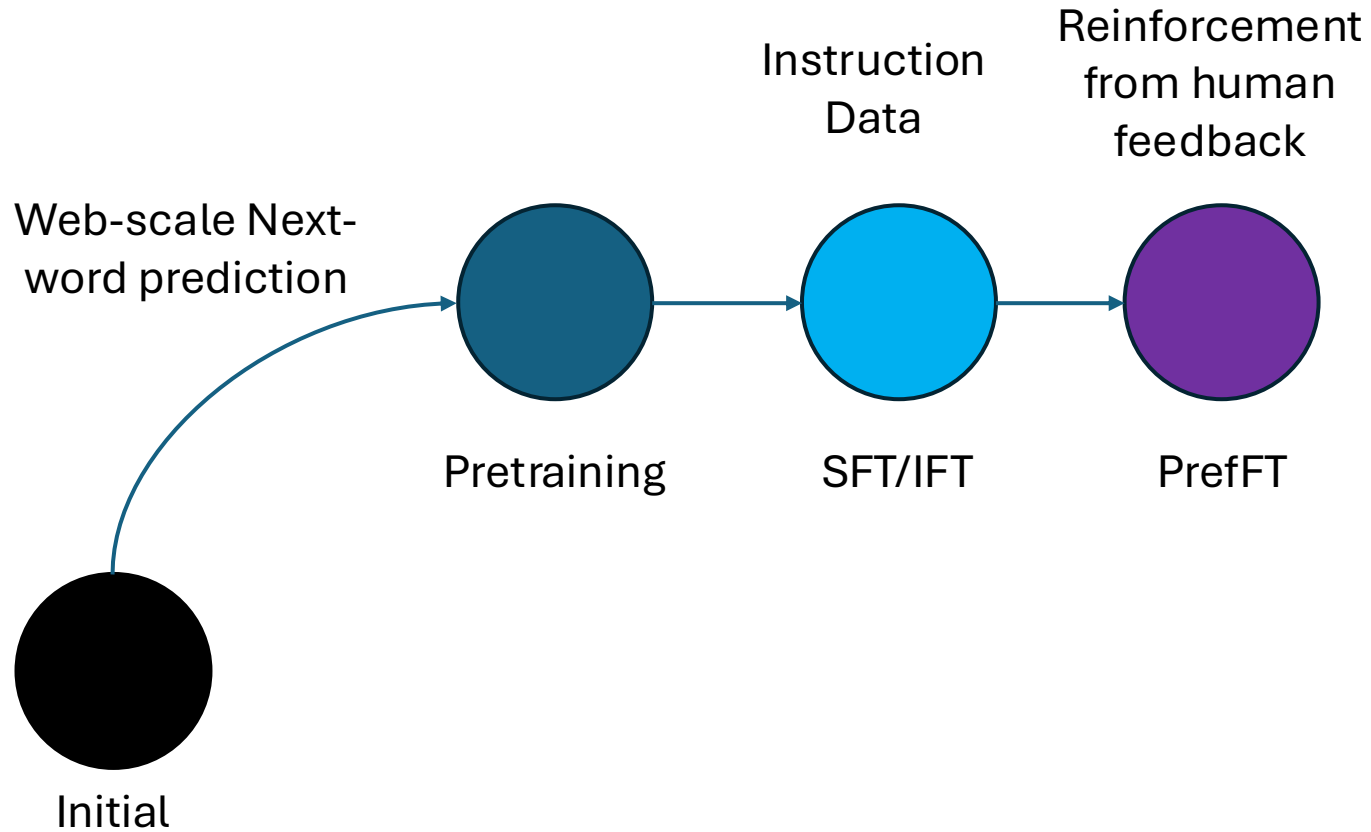
~\$430,000 on Amazon AWS!

Model	Layers	Width	Heads	Params	Data	Training
Transformer-Base	12	512	8	65M		8x P100 (12 hours)
Transformer-Large	12	1024	16	213M		8x P100 (3.5 days)
BERT-Base	12	768	12	110M	13 GB	
BERT-Large	24	1024	16	340M	13 GB	
XLNet-Large	24	1024	16	~340M	126 GB	512x TPU-v3 (2.5 days)
RoBERTa	24	1024	16	355M	160 GB	1024x V100 GPU (1 day)
GPT-2	12	768	?	117M	40 GB	
GPT-2	24	1024	?	345M	40 GB	
GPT-2	36	1280	?	762M	40 GB	
GPT-2	48	1600	?	1.5B	40 GB	
Megatron-LM	40	1536	16	1.2B	174 GB	64x V100 GPU
Megatron-LM	54	1920	20	2.5B	174 GB	128x V100 GPU
Megatron-LM	64	2304	24	4.2B	174 GB	256x V100 GPU (10 days)
Megatron-LM	72	3072	32	8.3B	174 GB	512x V100 GPU (9 days)

Generating images using diffusion models

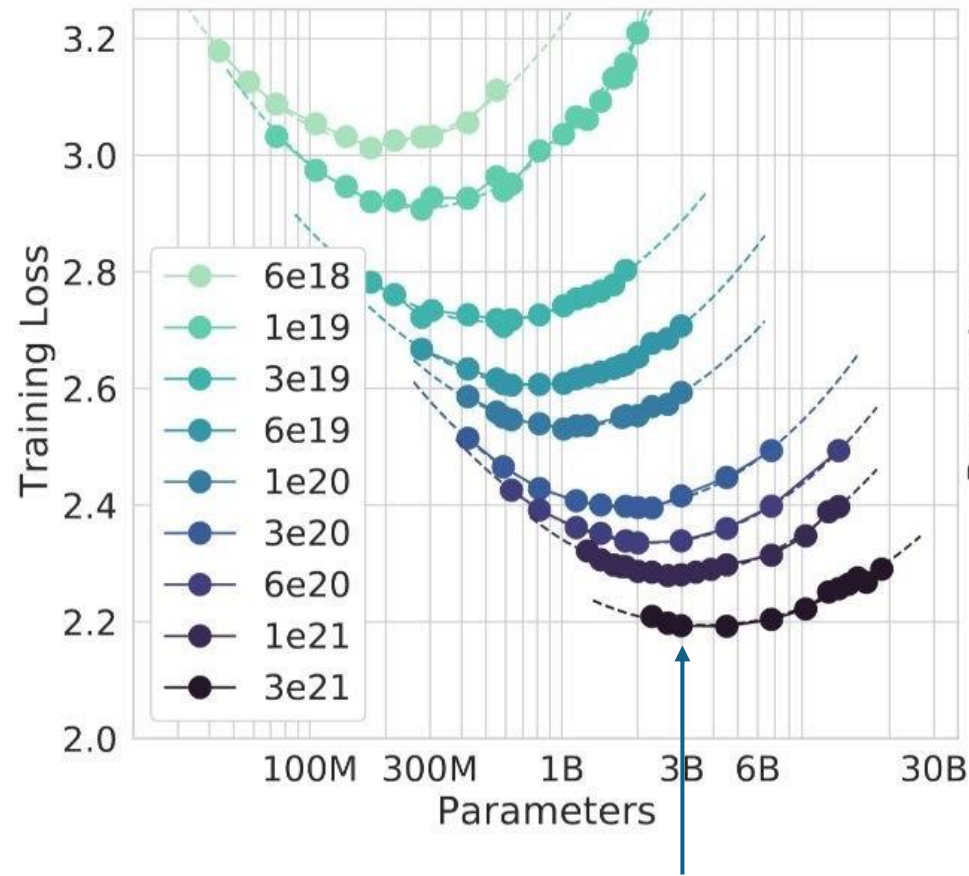


Basic steps for Chat GPT like model





Chinchilla law



We observe the trend that more data leads to lower loss, but without increasing the data, there is a limit

Given #tokens, each model has an optimal number of parameters



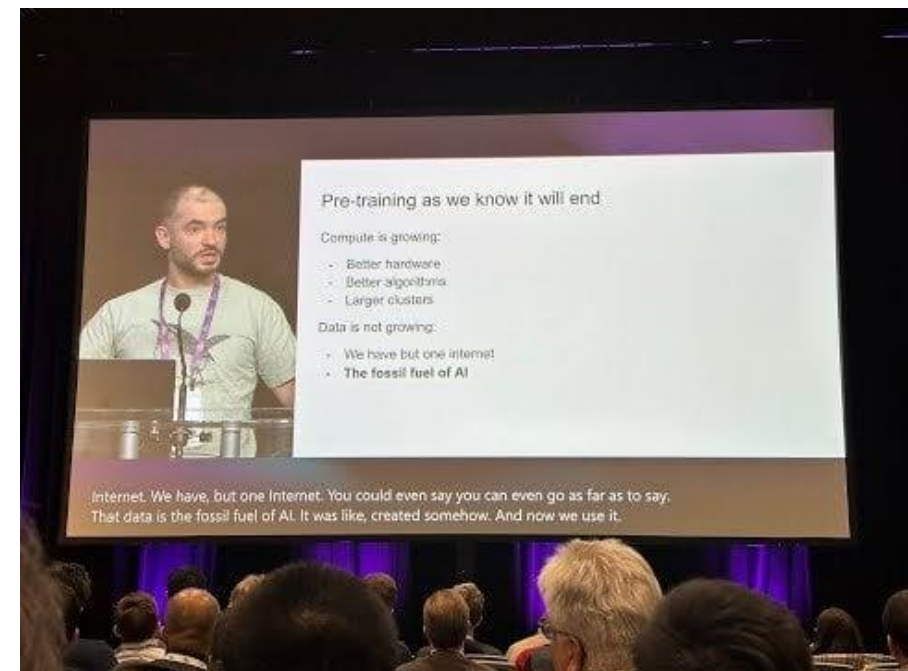
Ideal points

Parameters	FLOPs	FLOPs (in <i>Gopher</i> unit)	Tokens
400 Million	1.92e+19	1/29,968	8.0 Billion
1 Billion	1.21e+20	1/4,761	20.2 Billion
10 Billion	1.23e+22	1/46	205.1 Billion
67 Billion	5.76e+23	1	1.5 Trillion
175 Billion	3.85e+24	6.7	3.7 Trillion
280 Billion	9.90e+24	17.2	5.9 Trillion
520 Billion	3.43e+25	59.5	11.0 Trillion
1 Trillion	1.27e+26	221.3	21.2 Trillion
10 Trillion	1.30e+28	22515.9	216.2 Trillion



Can pre-training continue?

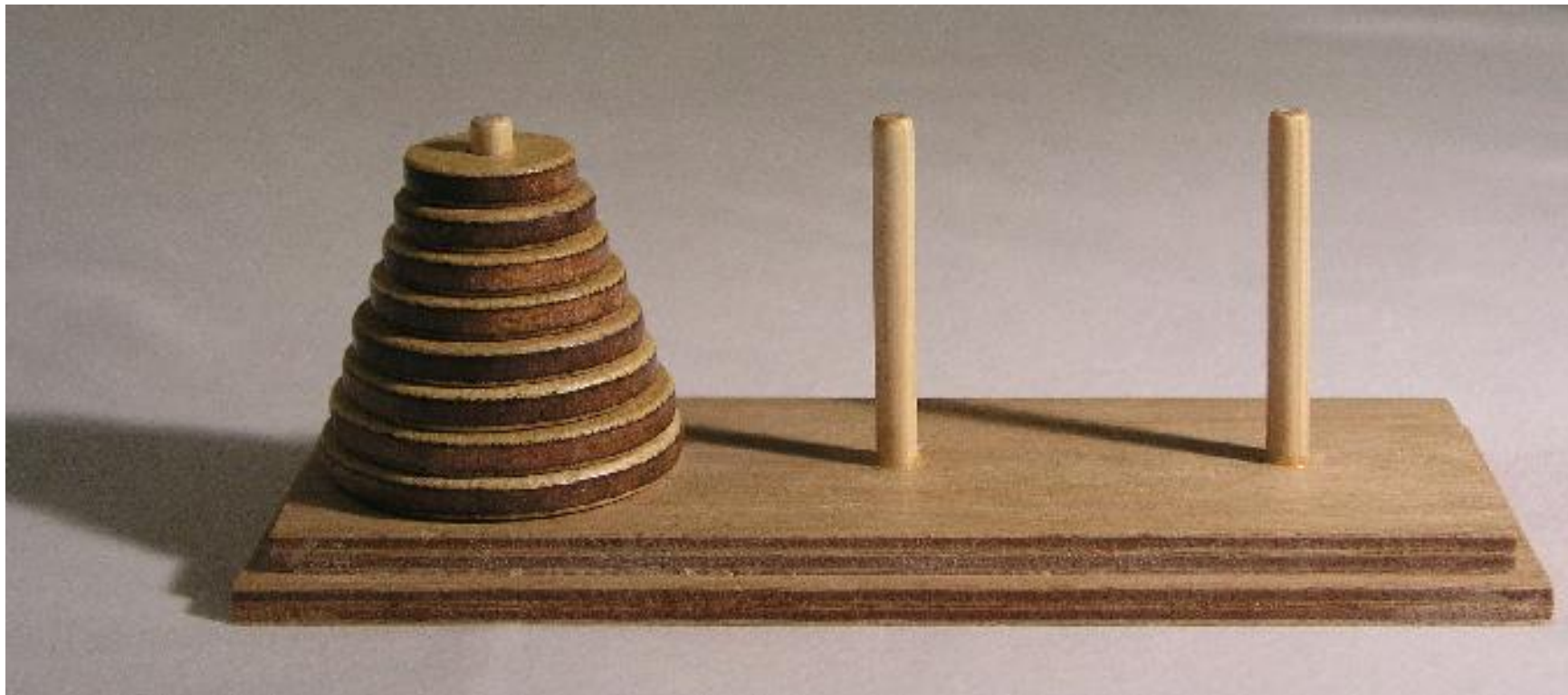
- Compute is growing:
 - Better hardware
 - Better algorithms
 - Larger clusters
- Data is not growing:
 - We have but one internet
 - The fossil fuel of AI
- **RL Post-training?**



Ilya Sutskever at Neurips'24

System 2: Problem solving

- Problem solving is the process of **using set of strategies** to find solutions to problems in an orderly manner.



Symbolic program execution for reasoning*

Query: How many muffins can each kid have for it to be fair?



Generated Code

```
def execute_command(image):  
    image_patch = ImagePatch(image)  
    muffin_patches = image_patch.find("muffin")  
    kid_patches = image_patch.find("kid")  
    return str(len(muffin_patches) // len(kid_patches))
```

Execution

```
muffin_patches =  
image_patch.find("muffin")
```



```
kid_patches =  
image_patch.find("kid")
```



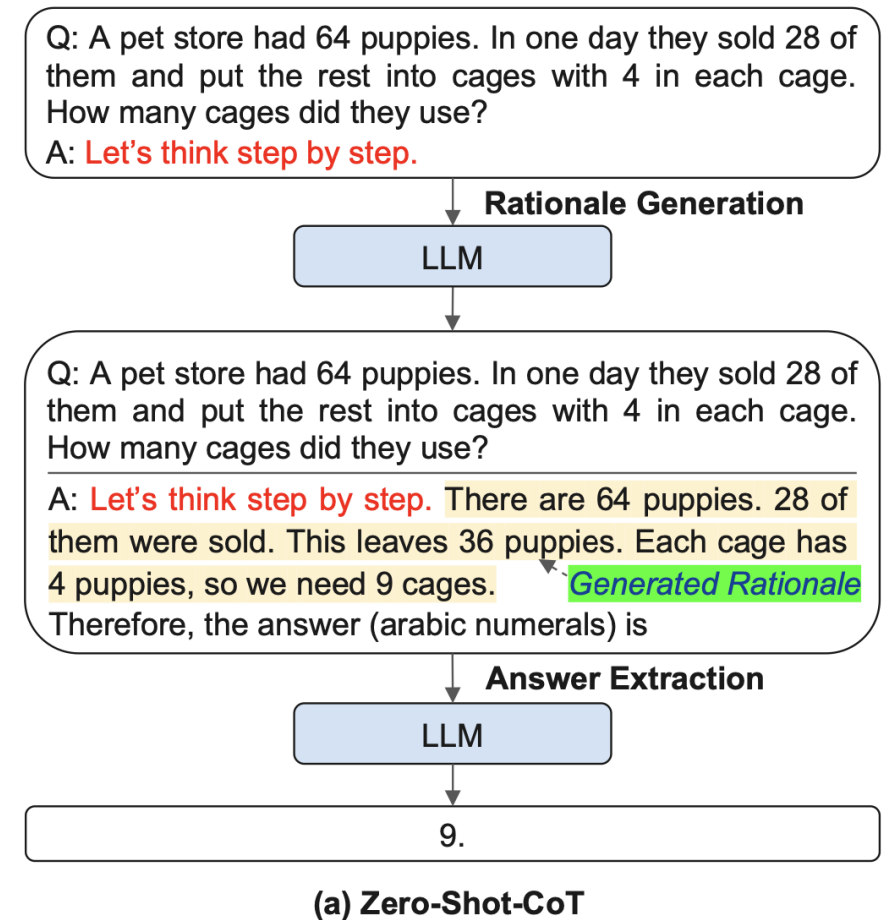
```
► len(muffin_patches)=8  
► len(kid_patches)=2
```

```
► 8//2 = 4
```

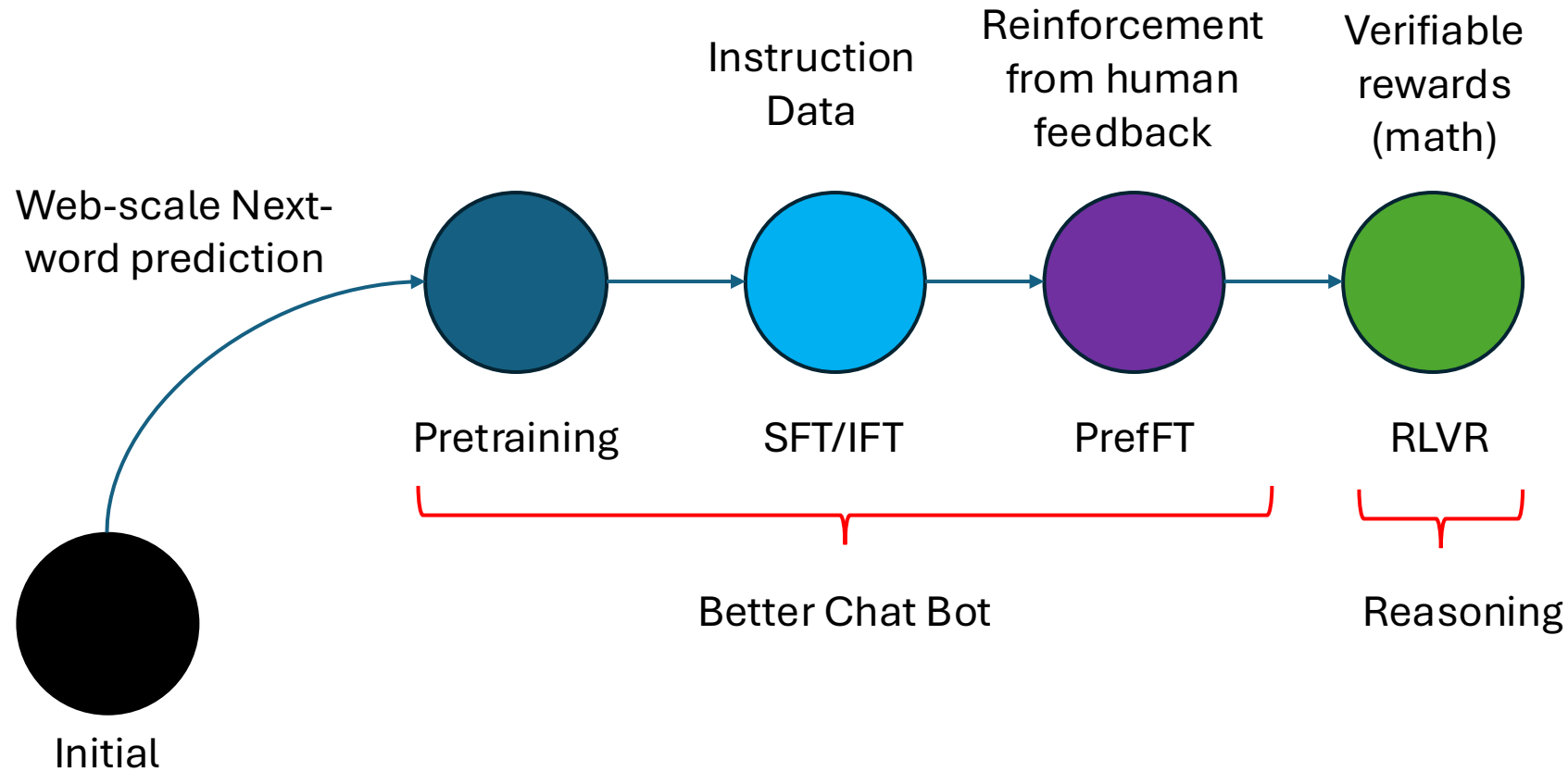
Result: 4

Reasoning models

- Introducing reasoning tokens, forcing the model to think
- Inspired by Chain-of-Thought (CoT) “think step by step” prompt before answering a question [1]



Basic steps for O1 like model



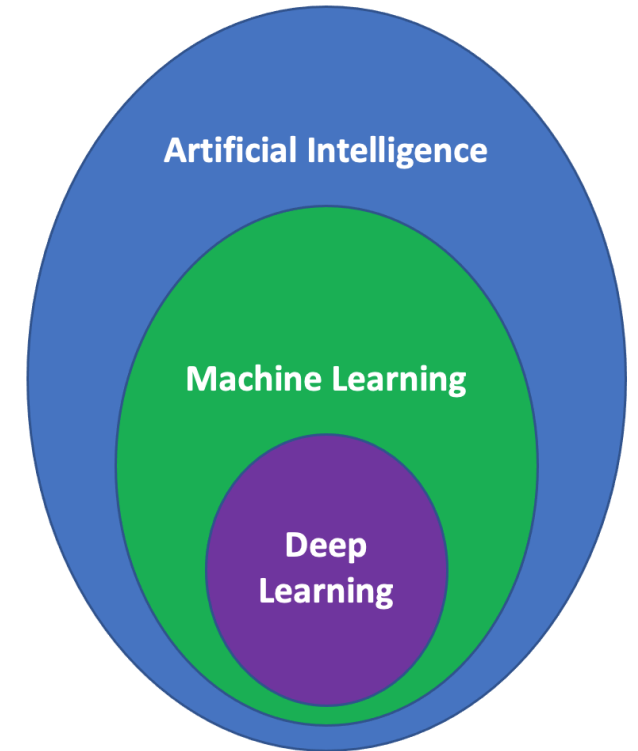
What RLVR allows

- Good for reasoning models training
 - reasoning supervision doesn't exist
- **Not mimicking** humans anymore
 - Exploring for a novel solution
- Challenges
 - Rewards are not good enough
 - Process rewards?
 - Very slow to train – bits of information for a long trajectory

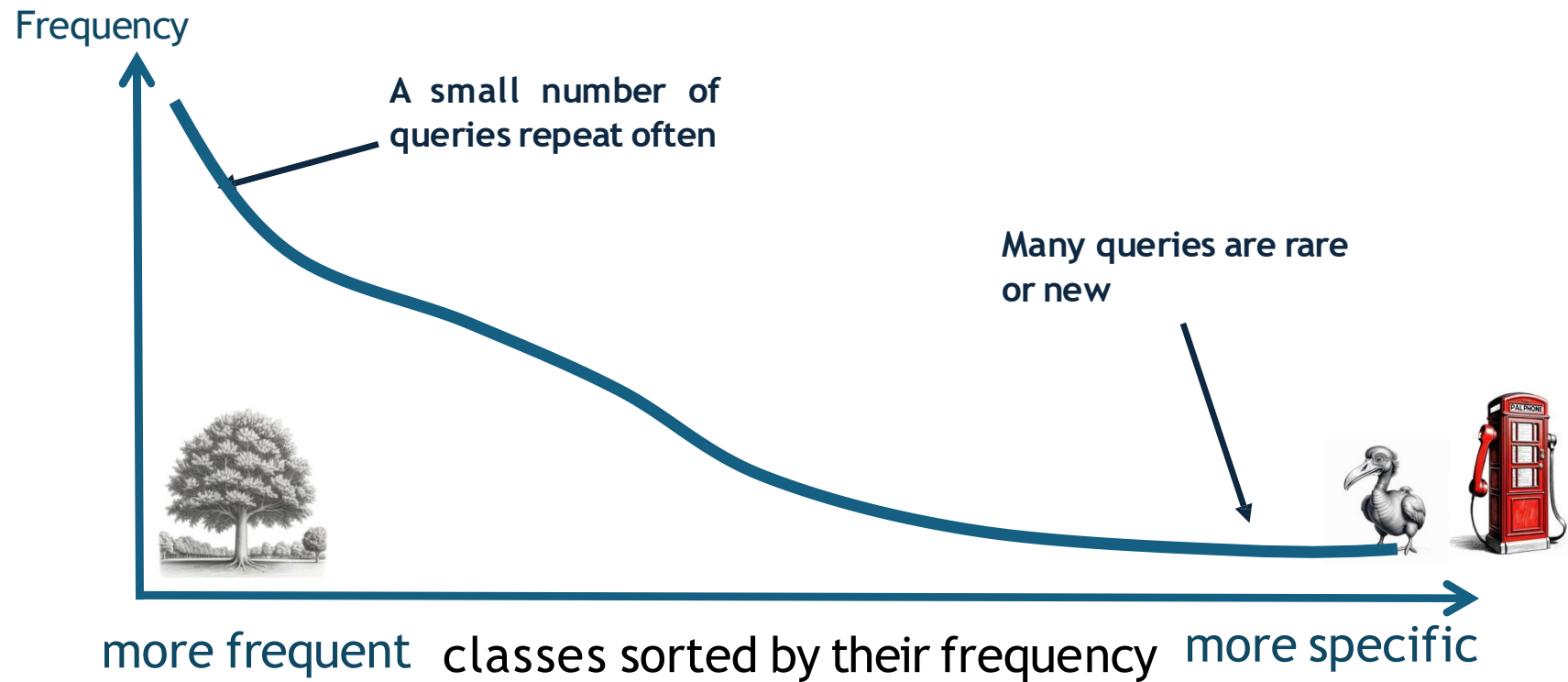
Takeaways:

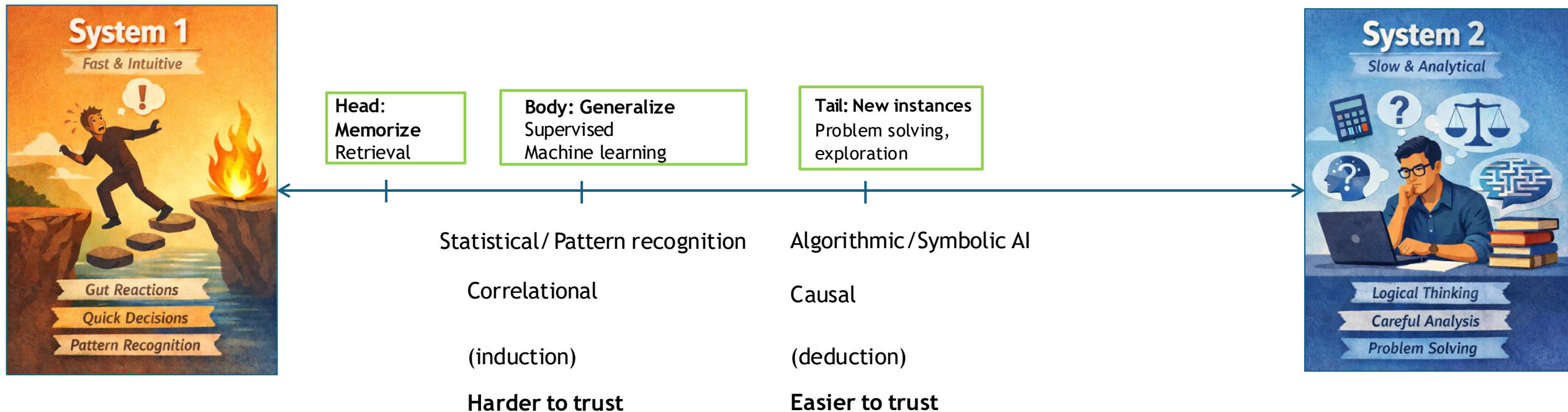
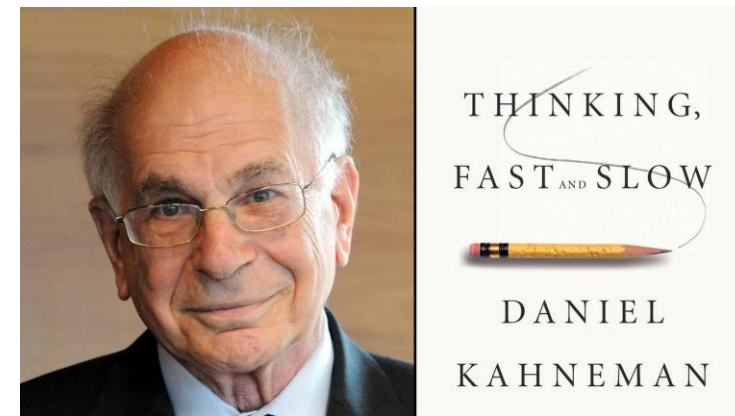
Reminder, what is AI, ML, DL

- **Artificial Intelligence (AI)** any technique that enables computers to mimic human intelligence (e.g., perception, comprehension, attention, problem solving, planning) from multiple modalities
- **Machine Learning (ML)** is a subset of AI that learn through use of data
- **Deep Learning (DL)** is a subset of ML that involves many parameters and large amounts of data



Problem with ML systems – they rely on data





LLMs w/ rational thinking

- Attention - asking LLMs where should I look at?^[1]
- Suppose I perceived object X and Y, does it makes sense?^[2]

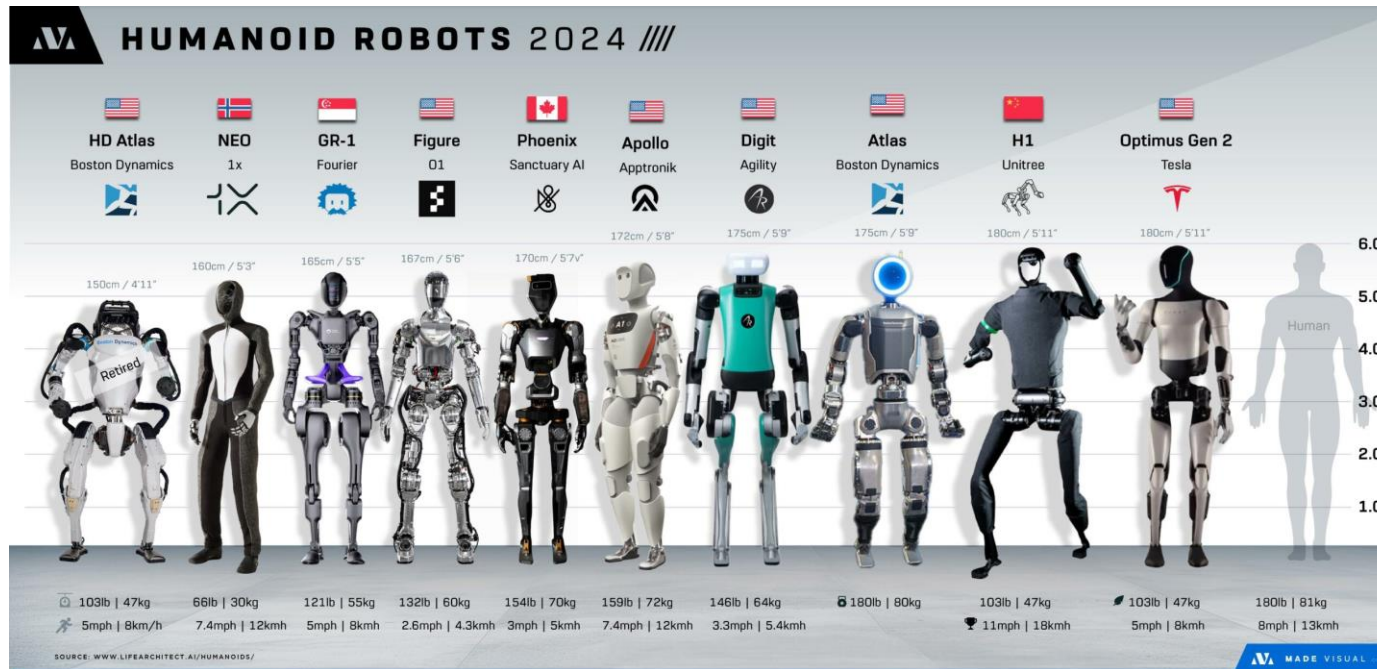


[1] - System 2 Attention (is something you might need too) (Weston et al.'23)

[2] - Breaking Common Sense: WHOOPS! (Bitton et al. ICCV'23)

System 2 and RL are also key for humanoids

- Embodied Agents – generalizing policies “What should I do next?”^[3,4]



[3] - MINEDOJO; (Fan et al. NeurIPS'22)

[4] - ThinkAct: Vision-Language-Action Reasoning via Reinforced Visual Latent Planning; Chi-Pin (NeurIPS'25)

Video from <https://www.nvidia.com/en-eu/ai/cosmos/>